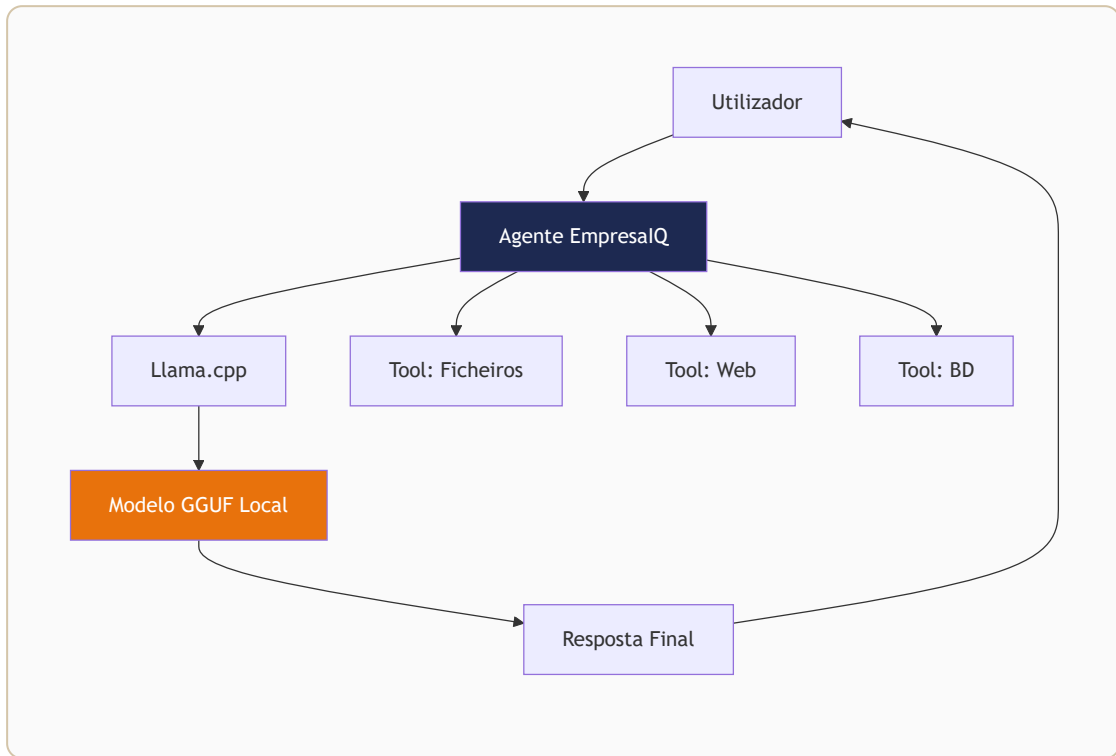


1. Introdução



O que vai aprender

Este guia mostra como criar um agente inteligente local baseado em modelos Open Source utilizando:

Tecnologia	Função
Phi-3-mini	Modelo de linguagem compacto e eficiente
Llama.cpp	Motor de inferência CPU otimizado
LangChain	Framework de construção de agentes
Python	Linguagem de programação
GGUF Quantizado	Formato de modelo comprimido

Tudo a funcionar diretamente no seu computador, **sem internet**, **sem APIs pagas** e **sem exposição de dados**.

Para quem é este guia

Este guia é ideal para:

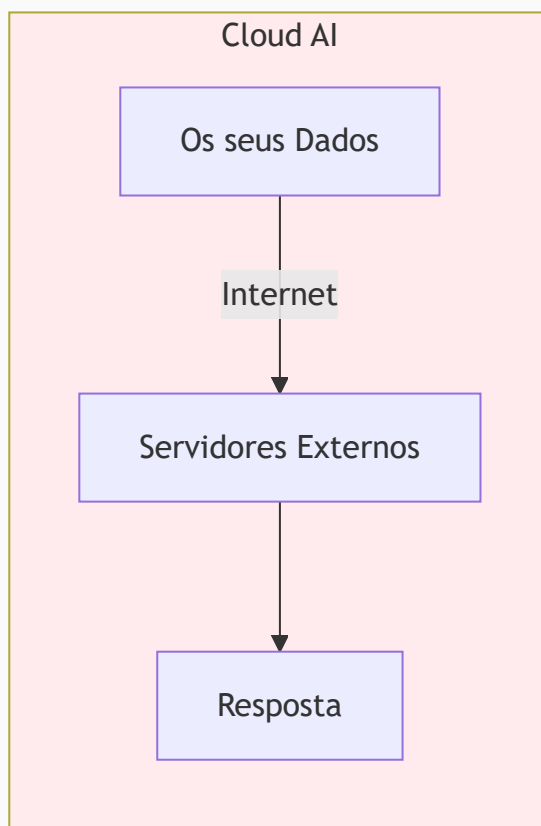
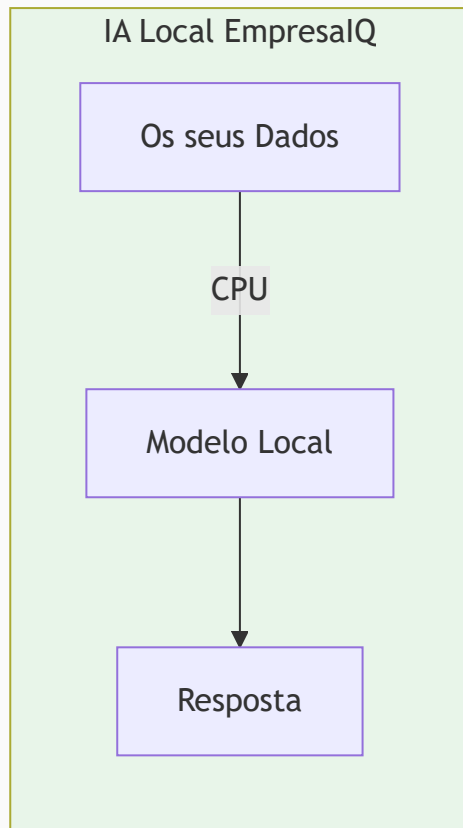
- **Empresas** que precisam de privacidade total
- **Consultores** que trabalham com dados sensíveis
- **Equipas de TI** que querem autonomia tecnológica
- **Programadores** que exploram IA local
- **Qualquer pessoa** com um PC normal e curiosidade

:::info Requisitos mínimos

- Windows 10/11, Linux ou macOS
- 8 GB RAM
- Python 3.11+
- 4 GB de espaço em disco livre :::

EmpresaIQ — O futuro da IA não pertence apenas à cloud. O futuro também corre localmente.

2. Porque usar IA Local



Privacidade Total

Os dados nunca saem do computador. Nenhum contrato, nenhuma proposta, nenhum dado de cliente é transmitido para servidores externos.

⚠ Riscos das APIs Cloud Ao usar ChatGPT, Claude ou Gemini via API:

- Os seus dados podem ser usados para treino de modelos
- Está sujeito a mudanças de política de privacidade
- Os dados atravessam múltiplos países e jurisdições ⚠

Sem Custos Mensais

Não existe pagamento de APIs. Compare:

Solução	Custo Estimado Mensal
GPT-4o API (uso moderado)	50€ – 500€
Claude API (uso moderado)	40€ – 400€
IA Local EmpresaIQ	0€

Após o setup inicial, o custo é **zero**.

Independência Total

Não depende da OpenAI, Anthropic ou outros fornecedores. Sem riscos de:

- Alterações de preços

- Descontinuação do serviço
- Limites de utilização
- Problemas de compliance

Velocidade Local

Sem latência de internet. A resposta depende apenas do CPU local — sem filas de espera em servidores remotos.

Segurança Empresarial

Ideal para sectores regulados:

Jurídico Finanças Saúde Governo

Contratos, pareceres, documentos confidenciais sem exposição.

Resumo das Vantagens

- ✓ Privacidade 100%
- ✓ Custo zero após setup
- ✓ Sem dependência de fornecedores
- ✓ Funciona offline
- ✓ Conformidade RGPD nativa
- ✓ Velocidade consistente
- ✓ Controlo total dos dados

3. Limitações de Hardware e Estratégia

0 Problema dos Modelos Grandes

Modelos populares exigem recursos enormes:

Modelo	RAM Necessária	GPU
Llama 3 70B	128 GB	Obrigatória
Mixtral 8x7B	64 GB	Obrigatória
DeepSeek V3	32 GB+	Obrigatória
Llama 3 8B (FP16)	16 GB	Recomendada

Estes modelos estão **fora de alcance** num PC normal.

A Estratégia Correta

A solução passa por três pilares:

1. Modelos Pequenos

Escolher modelos entre **2B e 4B parâmetros**. São surpreendentemente capazes para tarefas empresariais como:

- Responder perguntas sobre documentos
- Redigir emails e relatórios

- Analisar contratos
- Automatizar tarefas repetitivas

2. Quantização

Comprimir o modelo matematicamente para caber em menos RAM:

Modelo Original (FP16):	7 GB	✗	Não cabe
Modelo Q4_K_M:	2.2 GB	✓	Cabe facilmente

3. Inferência CPU com llama.cpp

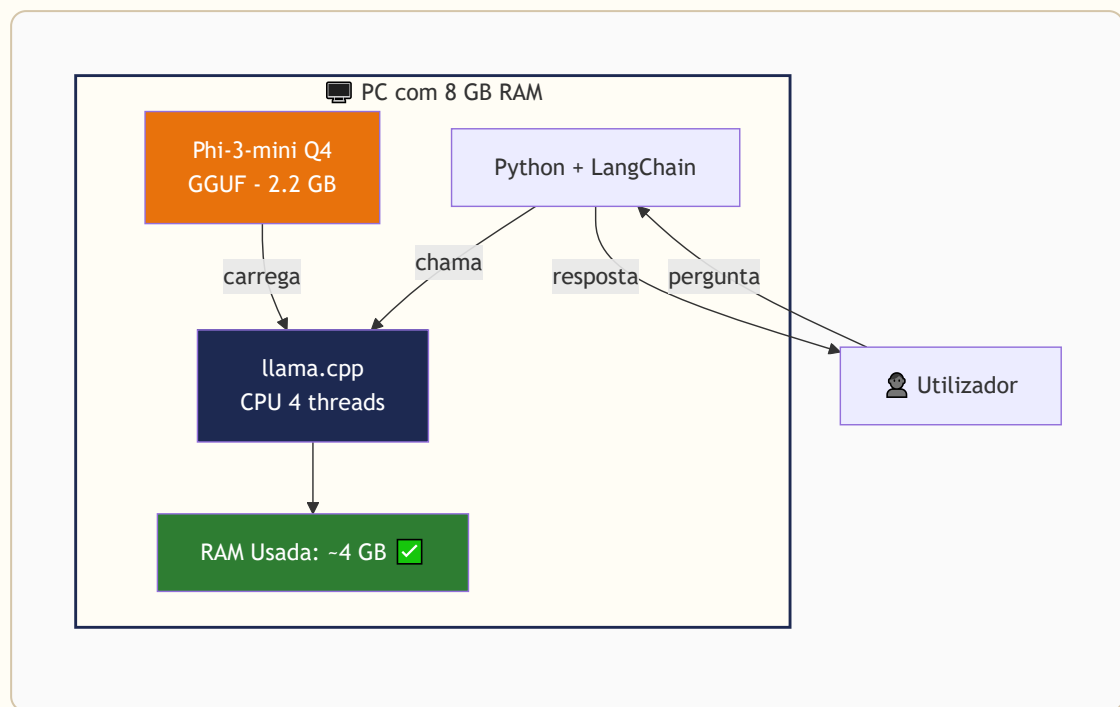
O llama.cpp é um motor de inferência escrito em C++ otimizado para CPU. Permite:

- Correr modelos GGUF directamente em CPU
- Utilizar todos os núcleos disponíveis
- Gerir a memória de forma eficiente

Comparação de Abordagem

✗	Abordagem Errada: "Quero correr Llama 3 70B no meu PC com 8 GB RAM" → Impossível
✓	Abordagem Correta: "Vou usar Phi-3-mini Q4 com llama.cpp" → Funciona perfeitamente

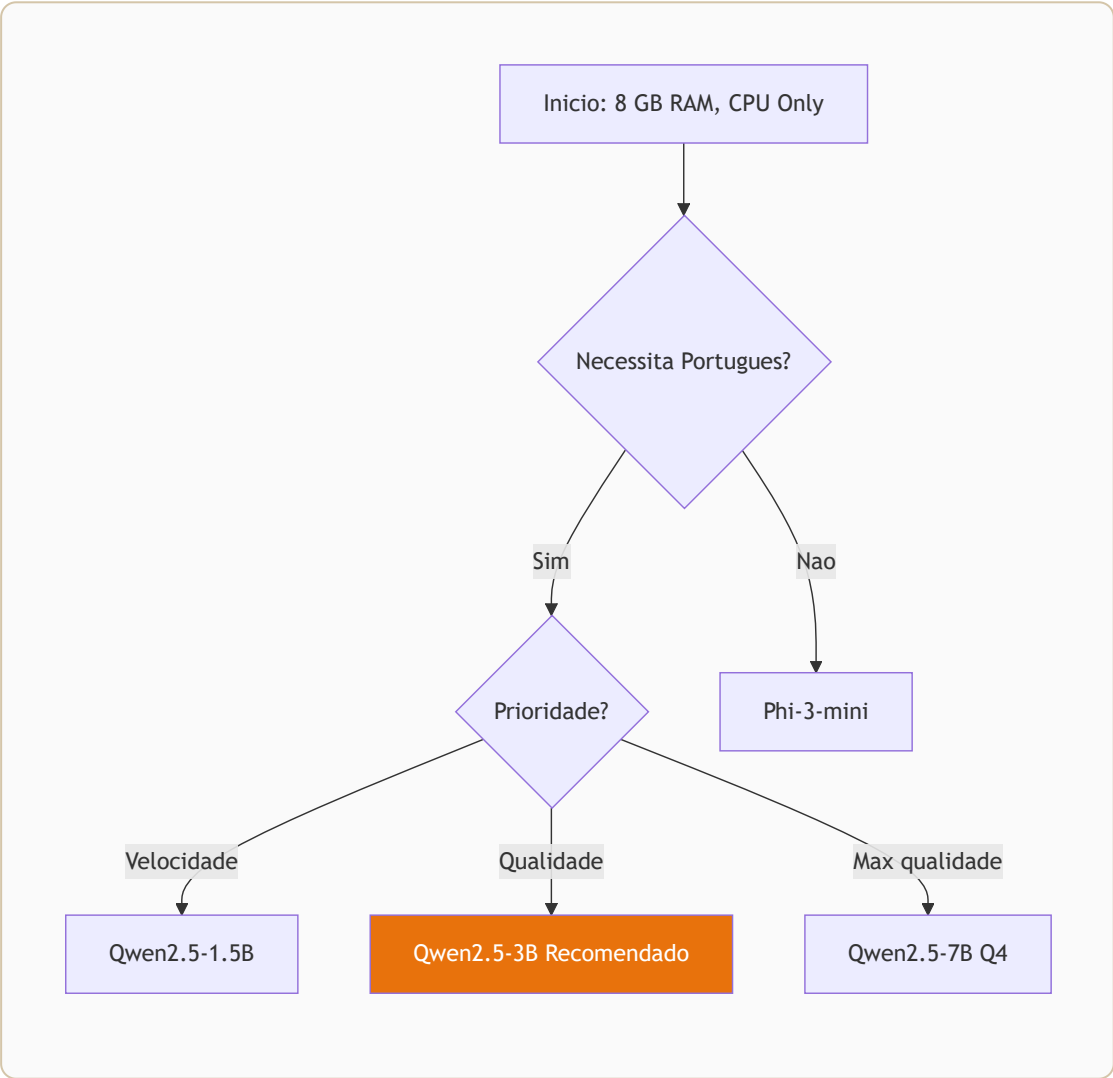
Arquitetura Final



:::tip Dica de Performance Feche aplicações desnecessárias antes de correr o agente. Cada GB de RAM livre melhora a velocidade de resposta. :::

Escolha do Modelo Ideal

```
mermaid flowchart TD
  A[Início: 8 GB RAM, CPU Only] --> B{Necessita Português?}
  B -->|Sim| C{Prioridade?}
  B -->|Não| D[Phi-3-mini]
  C -->|Velocidade| E[Qwen2.5-1.5B]
  C -->|Qualidade| F[Qwen2.5-3B  Recomendado]
  C -->|Máxima qualidade| G[Qwen2.5-7B Q4]
  style F fill:#E8720C,color:#fff
```



Melhor Escolha para 8 GB RAM: Phi-3-mini

O que é o Phi-3-mini?

O **Phi-3-mini-4k-instruct** é um modelo da Microsoft com 3.8B parâmetros. Apesar do tamanho reduzido, rivaliza com modelos muito maiores em tarefas práticas.

Foi treinado com foco em:

- Raciocínio lógico
- Seguimento de instruções
- Qualidade de escrita
- Eficiência computacional

Porquê Phi-3-mini e não outros?

Modelo	RAM (Q4)	Português	Velocidade	Raciocínio
Phi-3-mini	2.2 GB	 Excelente	 Rápido	★ ★ ★ ★ ★
TinyLlama 1.1B	0.8 GB	 Fraco	 Muito rápido	★ ★
Llama 3 8B Q2	3.5 GB	 Bom	 Lento	★ ★ ★ ★
Mistral 7B Q2	3 GB	 Bom	 Lento	★ ★ ★ ★
Gemma 2B	1.5 GB	 Médio	 Rápido	★ ★ ★

Phi-3-mini é o ponto óptimo entre qualidade, velocidade e consumo de RAM.

Estimativas Reais de Consumo

Sistema Operativo:	~2.0 GB RAM
Phi-3-mini Q4_K_M:	~2.2 GB RAM
Python + LangChain:	~0.5 GB RAM
Total:	~4.7 GB RAM
Sobra para o SO:	~3.3 GB RAM 

Variantes Disponíveis

O Phi-3-mini existe em duas versões de contexto:

Variante	Contexto	Uso
<code>phi-3-mini-4k-instruct</code>	4.096 tokens	Recomendado — tarefas gerais
<code>phi-3-mini-128k-instruct</code>	128.000 tokens	Documentos longos (mais lento)

:::info Recomendação Para um agente empresarial com 8 GB RAM, use sempre a versão **4k**. Contexto de 128k exige muito mais memória e é muito mais lento em CPU. :::

Como Descarregar

O modelo está disponível no **Hugging Face** em formato GGUF, já quantizado:

Repositório:

Ficheiro:

Tamanho:

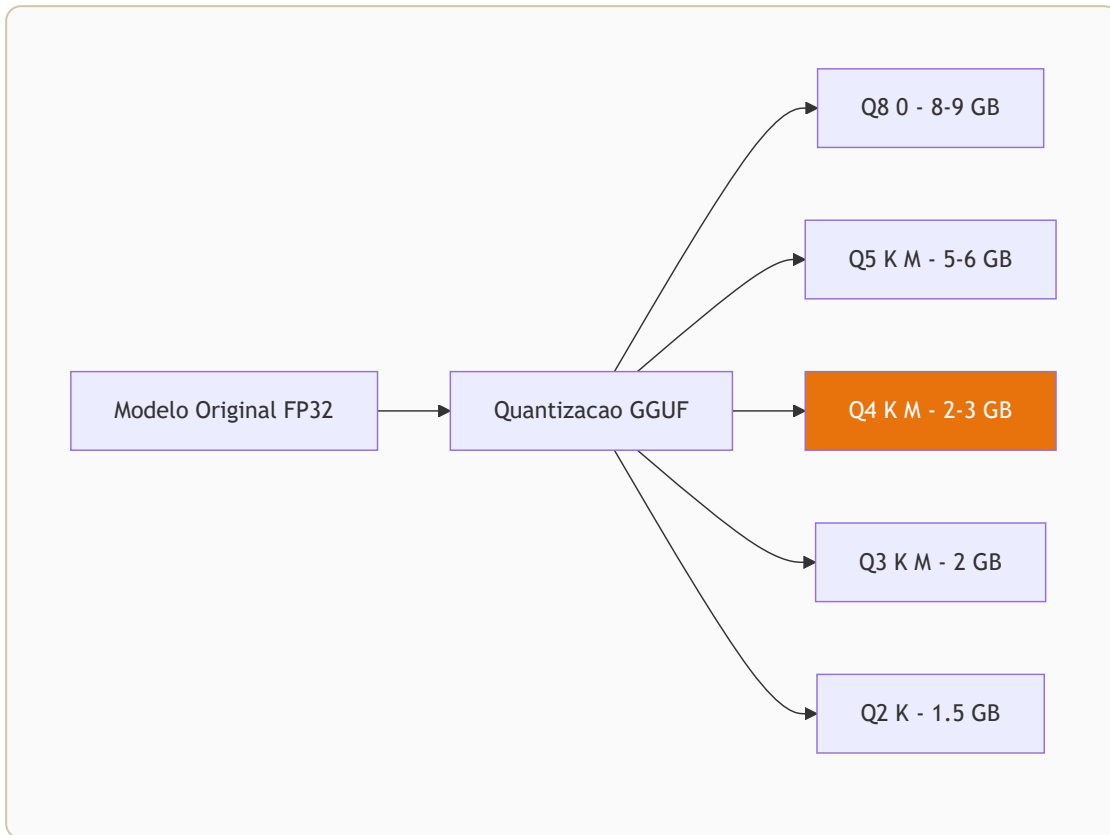
bartowski/Phi-3-mini-4k-instruct-GGUF

Phi-3-mini-4k-instruct-Q4_K_M.gguf

~2.2 GB

Na secção **9. Download do Modelo** encontra as instruções completas.

O que é GGUF e Quantização



GGUF — O Formato Certo para CPU

GGUF (GPT-Generated Unified Format) é o formato padrão para modelos otimizados para CPU. Foi criado pela equipa do llama.cpp e tornou-se o standard da comunidade.

Vantagens do GGUF

- ✓ Carregamento rápido (mmap – não copia para RAM desnecessariamente)
- ✓ Suporte a quantização múltipla no mesmo ficheiro
- ✓ Metadados integrados (tokenizer, arquitectura, etc.)
- ✓ Compatível com llama.cpp, Ollama, LM Studio, GPT4All
- ✓ Cross-platform: Windows, Linux, macOS

Estrutura de um ficheiro GGUF

```
modelo.gguf
├─ Metadados do modelo
├─ Vocabulário (tokenizer)
├─ Arquitetura (número de camadas, etc.)
└─ Pesos (quantizados)
```

Quantização — Comprimir sem Perder Qualidade

A quantização reduz a precisão matemática dos pesos do modelo para ocupar menos memória.

Analogia Simples

Imagine uma fotografia:

- **Original (FP32):** 100 MB — qualidade máxima
- **JPEG 90%:** 15 MB — quase indistinguível
- **JPEG 60%:** 4 MB — boa para uso geral
- **JPEG 30%:** 1 MB — aceitável para miniaturas

Com modelos de IA acontece o mesmo:

Tipo	Bits/Peso	Qualidade	RAM (Phi-3-mini)	Velocidade
FP32	32 bits	Perfeita	~15 GB	Muito lenta
FP16	16 bits	Excelente	~7.6 GB	Lenta
Q8_0	8 bits	Muito boa	~4 GB	Moderada
Q4_K_M	4 bits	Boa	~2.2 GB	⚡ Rápida
Q3_K_M	3 bits	Aceitável	~1.8 GB	⚡⚡ Muito rápida
Q2_K	2 bits	Limitada	~1.3 GB	⚡⚡⚡ Máxima

A Nomenclatura GGUF

Q4_K_M

```

| | |
| | └─ M = Medium (equilíbrio qualidade/tamanho)
| └─── K = K-quant (algoritmo de quantização melhorado)
└──── 4 = 4 bits por peso

```

Outras variantes comuns:

- Q4_K_S — Small (menor, ligeiramente pior qualidade)
- Q4_K_L — Large (melhor qualidade, mais RAM)
- Q5_K_M — 5 bits, melhor qualidade que Q4

Qual Escolher?

8 GB RAM disponível:

Recomendação: Q4_K_M

- Qualidade praticamente idêntica ao FP16
- 2.2 GB para Phi-3-mini
- Velocidade boa em CPU
- Ponto ótimo qualidade/recursos



DICA

Se tiver 6 GB RAM disponíveis para o modelo, experimente

`Q5_K_M` — melhor qualidade com apenas mais 300 MB.

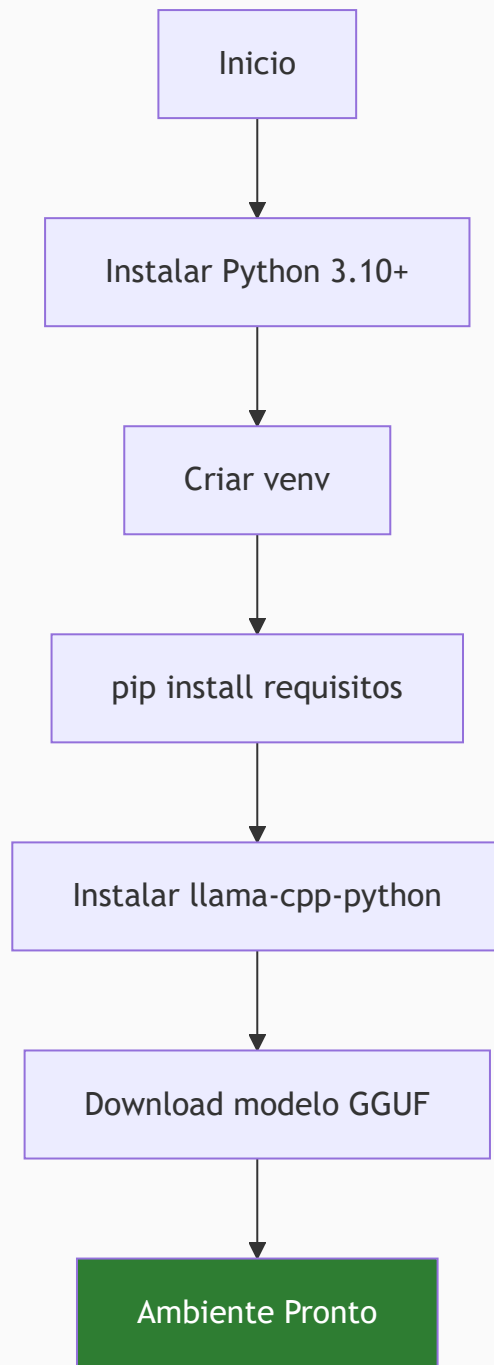
Onde Encontrar Modelos GGUF

Os melhores repositórios de modelos GGUF quantizados:

- **bartowski** — Quantizações de alta qualidade
- **TheBloke** — Biblioteca enorme de modelos
- **lmstudio-community** — Optimizados para uso local

Instalação do Ambiente

```
mermaid flowchart TD
    A[🖥️ Início da Instalação] --> B[Python 3.10+]
    B --> C[Criar venv]
    C --> D[pip install langchain]
    D --> E[pip install llama-cpp-python]
    E --> F[pip install gradio]
    F --> G[Download modelo GGUF]
    G --> H[✅ Ambiente Pronto]
    style H fill:#2E7D32,color:#fff
```



1. Instalar Python 3.11

Windows

1. Aceda a python.org/downloads

2. Descarregue **Python 3.11.x** (não use 3.12+ por compatibilidade com llama-cpp-python)
3. Execute o instalador
4. **IMPORTANTE:** Marque  "Add Python to PATH"

Verifique a instalação:

```
python --version  
# Python 3.11.x
```

Linux (Ubuntu/Debian)

```
sudo apt update  
sudo apt install python3.11 python3.11-venv python3.11-pip -y
```

macOS

```
brew install python@3.11
```

2. Criar a Pasta do Projecto

```
mkdir empresaiq-agent  
cd empresaiq-agent
```

3. Criar Ambiente Virtual

Um ambiente virtual isola as dependências do projecto:

```
# Windows  
python -m venv venv  
venv\Scripts\activate
```

```
# Linux / macOS
python3.11 -m venv venv
source venv/bin/activate
```

Quando activado, o terminal mostra:

```
(venv) C:\empresaiq-agent>
```

:::warning Importante **Sempre active o ambiente virtual** antes de instalar pacotes ou executar o agente. Caso contrário, as dependências serão instaladas globalmente. :::

4. Estrutura Final do Projecto

Após seguir todos os capítulos, a estrutura será:

```
empresaiq-agent/
|
├─ venv/                ← Ambiente virtual Python
├─ agente_local.py      ← Agente principal
├─ tools.py             ← Ferramentas do agente
├─ requirements.txt     ← Dependências
└─ Phi-3-mini-4k-instruct-Q4_K_M.gguf ← Modelo IA
```

5. Verificar Ferramentas Necessárias

```
# Verificar Python
python --version

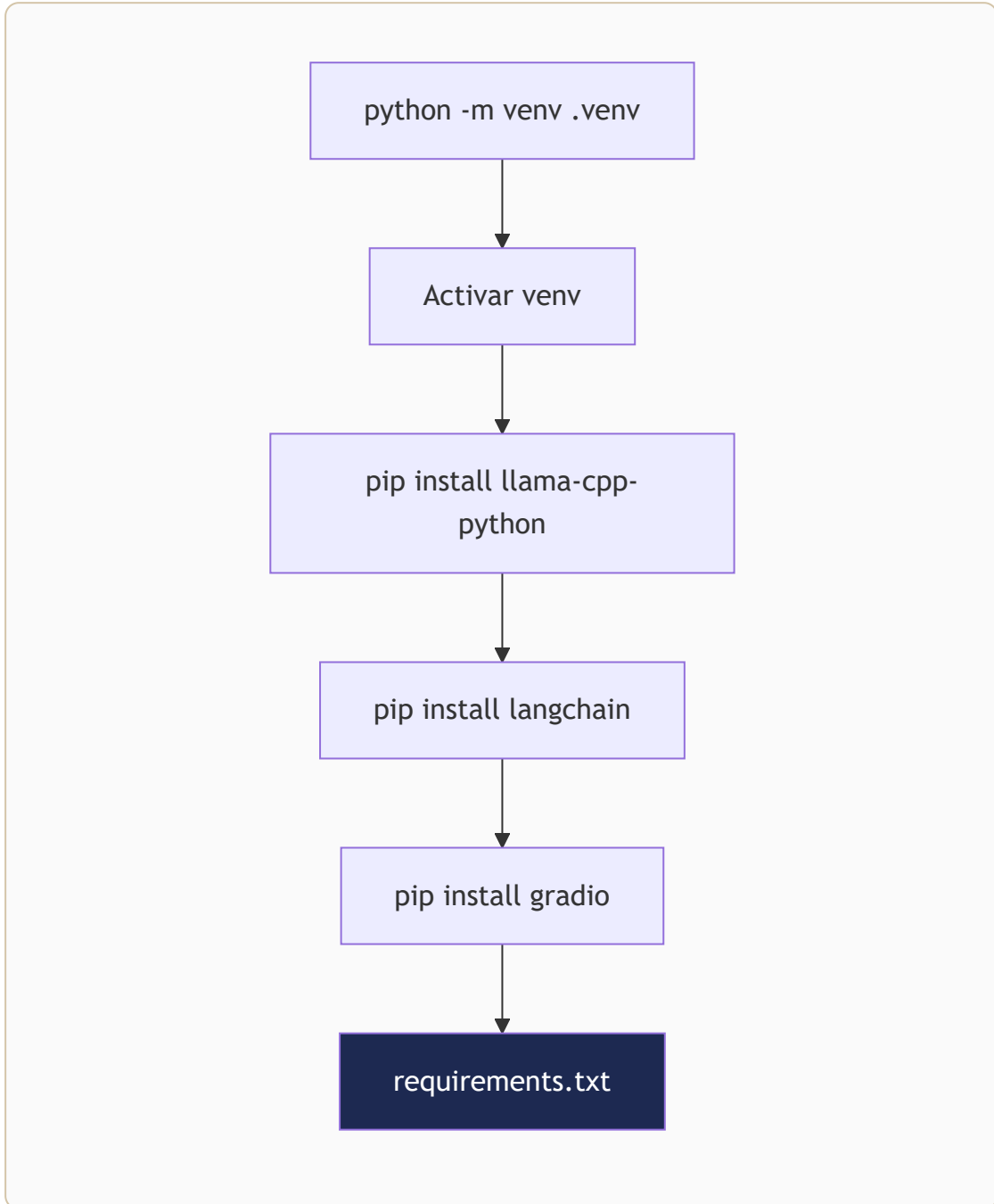
# Verificar pip
pip --version

# Verificar espaço em disco (necessário ~5 GB)
# Windows
Get-PSDrive C
```

```
# Linux  
df -h /
```

:::tip Dica Se tiver problemas com permissões no Windows, execute o terminal como **Administrador**. :::

Configuração do Python



1. Criar o ficheiro requirements.txt

Dentro da pasta `empresaiq-agent`, crie o ficheiro `requirements.txt`:

requirements.txt

```
llama-cpp-python==0.2.76
langchain==0.1.16
langchain-community==0.0.34
requests==2.31.0
```

O que cada pacote faz

Pacote	Versão	Função
<code>llama-cpp-python</code>	0.2.76	Binding Python para llama.cpp — corre o modelo GGUF
<code>langchain</code>	0.1.16	Framework para construir agentes e chains
<code>langchain-community</code>	0.0.34	Integrações comunitárias (inclui <code>LlamaCpp</code> LLM)
<code>requests</code>	2.31.0	Chamadas HTTP para ferramentas externas

2. Instalar as Dependências

Com o ambiente virtual activado:

```
pip install -r requirements.txt
```

caution Instalação do llama-cpp-python Este pacote **compila código C++** durante a instalação. Pode demorar 5-15 minutos dependendo do CPU.

Windows: Precisa de ter o **Microsoft C++ Build Tools** instalado.

Linux: Instale primeiro:

```
sudo apt install build-essential cmake -y
```

⋮

3. Instalação Alternativa — Pré-compilado (Windows)

Para evitar a compilação no Windows, use a versão pré-compilada:

```
pip install llama-cpp-python --extra-index-url  
https://abetlen.github.io/llama-cpp-python/whl/cpu
```

Depois instale os restantes:

```
pip install langchain==0.1.16 langchain-community==0.0.34  
requests==2.31.0
```

4. Verificar a Instalação

```
# Teste rápido – corra no terminal Python  
python -c "from llama_cpp import Llama; print('llama-cpp-  
python OK')"  
python -c "from langchain.agents import tool;  
print('LangChain OK')"
```

Se ambos mostrarem `OK`, está pronto para o próximo passo.

5. Resolução de Problemas Comuns

Erro: Microsoft Visual C++ 14.0 is required

Instale o **Microsoft C++ Build Tools** e escolha "Desktop development with C++".

Erro: cmake not found

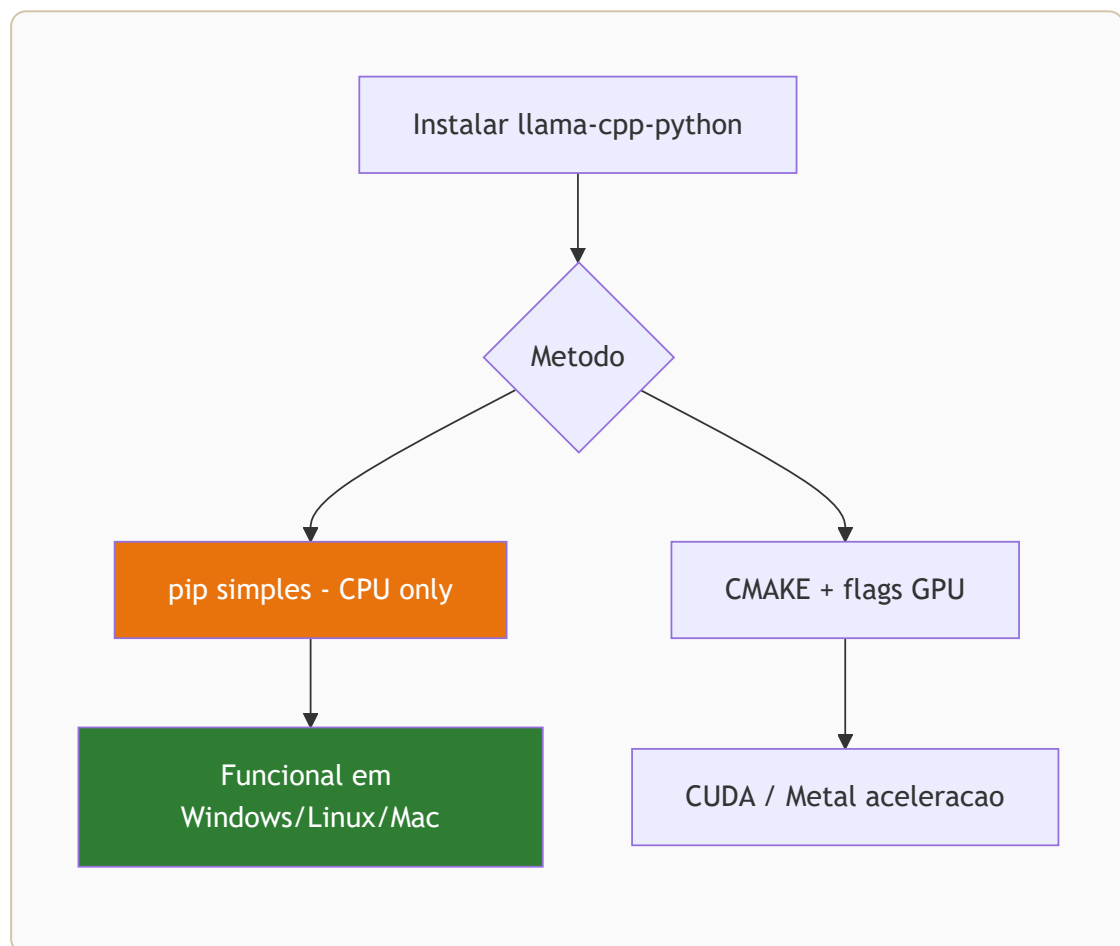
```
pip install cmake
```

Erro de versão Python

```
# Verifique que está a usar Python 3.11  
python --version  
# Se mostrar 3.12+, instale 3.11 e recriar o venv
```

Instalação do Llama.cpp

```
mermaid graph LR
  A[llama-cpp-python] --> B[Método de instalação]
  B --> C[pip install simples]
  B --> D[Build com otimizações]
  C --> E[CPU básico]
  D --> F[CMAKE_ARGS=-DLLAMA_AVX2=on]
  F --> G[⚡ CPU otimizado]
  style G fill:#E8720C,color:#fff
```



O que é o llama.cpp?

O llama.cpp é uma biblioteca escrita em C++ criada por **Georgi Gerganov**. É o coração da inferência local em CPU.

Características principais

- ✓ Escrito em C++ puro – performance máxima
- ✓ Suporte a múltiplos threads CPU
- ✓ Gestão eficiente de memória (mmap)
- ✓ Compatível com todos os modelos GGUF
- ✓ Cross-platform: Windows, Linux, macOS
- ✓ Zero dependências externas

Como é Instalado

O `llama-cpp-python` já inclui e compila o motor `llama.cpp` automaticamente.

```
pip install llama-cpp-python==0.2.76
```

Durante a instalação vai ver:

```
Building wheels for collected packages: llama-cpp-python
  Building wheel for llama-cpp-python (pyproject.toml) ...
    running build_ext
      cmake --build ...          ← A compilar llama.cpp
      ...
  Successfully built llama-cpp-python
```

Optimizações CPU por Plataforma

Windows — AVX2 (CPUs modernos Intel/AMD)

```
set CMAKE_ARGS="-DLLAMA_AVX2=on"
pip install llama-cpp-python==0.2.76 --force-reinstall
```

Linux — AVX2

```
CMAKE_ARGS="-DLLAMA_AVX2=on" pip install llama-cpp-  
python==0.2.76
```

Como verificar suporte AVX2

```
# Windows (PowerShell)  
Get-WmiObject -Class Win32_Processor | Select-Object -  
ExpandProperty Name  
  
# Linux  
grep -m1 avx2 /proc/cpuinfo
```

:::info Processadores com AVX2 A maioria dos processadores Intel Core a partir de 2013 (Haswell) e AMD Ryzen têm suporte AVX2. Com AVX2 activo, a velocidade de inferência melhora 30-50%. :::

Testar o Motor

```
from llama_cpp import Llama  
  
# Teste sem modelo completo – apenas verifica a importação  
print("llama.cpp versão:", Llama.__version__ if  
hasattr(Llama, '__version__') else "OK")
```

Como Funciona Internamente

```
Pergunta do Utilizador  
↓  
Python (LangChain)  
↓  
llama-cpp-python (binding)  
↓  
llama.cpp (C++)  
↓  
Modelo GGUF em disco  
↓
```

Inferência multi-thread em CPU



Tokens gerados

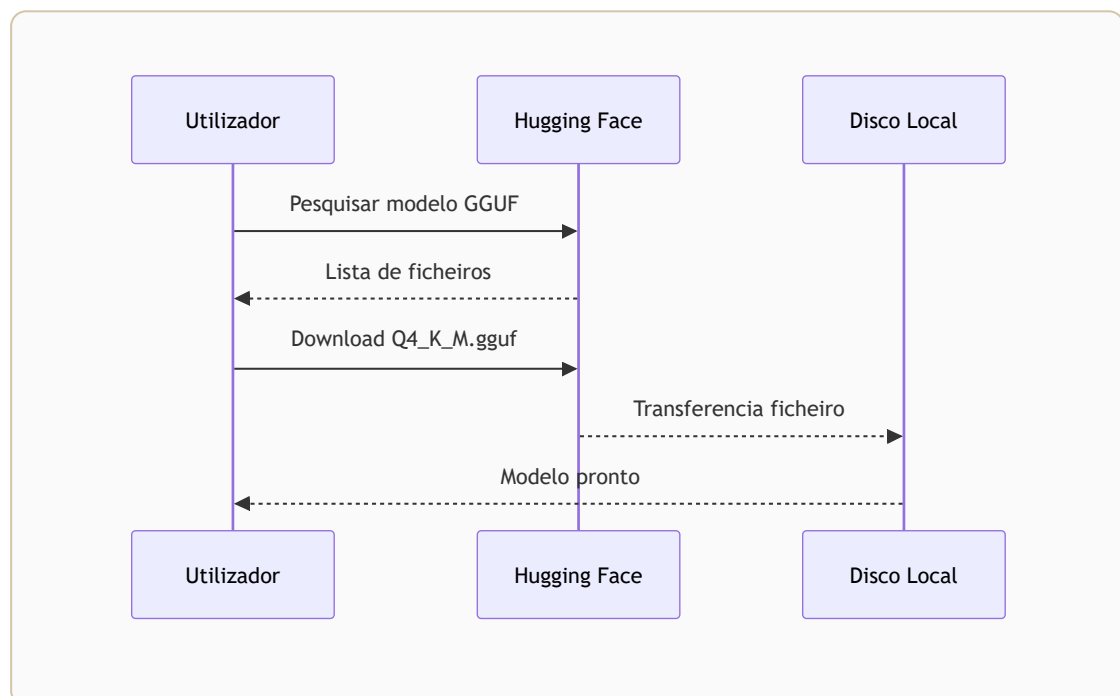


Resposta em texto

O processo de inferência usa **todos os núcleos do CPU** configurados via `n_threads`, maximizando a velocidade disponível no hardware.

Download do Modelo Phi-3-mini

```
mermaid flowchart TD
    A[Hugging Face Hub] --> B[Escolher modelo GGUF]
    B --> C{Método de download}
    C --> D[huggingface-cli download]
    C --> E[wget / Browser]
    D --> F[./models/modelo.gguf]
    E --> F
    F --> G[✅ Modelo Pronto]
    style G fill:#2E7D32,color:#fff
```



Modelo Recomendado

Ficheiro: Phi-3-mini-4k-instruct-Q4_K_M.gguf
Tamanho: ~2.2 GB
Fonte: Hugging Face

Método 1 — Download Directo (Browser)

1. Aceda a: huggingface.co/bartowski/Phi-3-mini-4k-instruct-GGUF
 2. Clique em "Files and versions"
 3. Localize `Phi-3-mini-4k-instruct-Q4_K_M.gguf`
 4. Clique no ícone de download 
 5. Mova o ficheiro para a pasta `empresaiq-agent/`
-

Método 2 — Linha de Comandos (huggingface-hub)

```
pip install huggingface-hub

python -c "
from huggingface_hub import hf_hub_download
hf_hub_download(
    repo_id='bartowski/Phi-3-mini-4k-instruct-GGUF',
    filename='Phi-3-mini-4k-instruct-Q4_K_M.gguf',
    local_dir='.'
)
print('Download concluído!')
"
```

Método 3 — wget / curl

```
# Linux / macOS
wget https://huggingface.co/bartowski/Phi-3-mini-4k-instruct-GGUF/resolve/main/Phi-3-mini-4k-instruct-Q4_K_M.gguf

# Windows PowerShell
Invoke-WebRequest -Uri
"https://huggingface.co/bartowski/Phi-3-mini-4k-instruct-
```

```
GGUF/resolve/main/Phi-3-mini-4k-instruct-Q4_K_M.gguf" -  
OutFile "Phi-3-mini-4k-instruct-Q4_K_M.gguf"
```

Verificar o Ficheiro

Após o download, verifique:

```
# Windows  
Get-Item "Phi-3-mini-4k-instruct-Q4_K_M.gguf" | Select-  
Object Name, Length  
  
# Linux / macOS  
ls -lh Phi-3-mini-4k-instruct-Q4_K_M.gguf
```

Deve mostrar aproximadamente 2.2 GB.

Estrutura Final do Projecto

```
empresaiq-agent/  
|  
├─ venv/  
├─ agente_local.py      ← (próximo capítulo)  
├─ tools.py             ← (próximo capítulo)  
├─ requirements.txt  
└─ Phi-3-mini-4k-instruct-Q4_K_M.gguf  ✅ Aqui!
```

Modelos Alternativos

Se preferir explorar outras opções:

Modelo	Ficheiro GGUF	RAM	Notas
Phi-3-mini (recomendado)	Phi-3-mini-4k-instruct-Q4_K_M.gguf	2.2 GB	✓ Melhor escolha
Gemma 2B	gemma-2b-it-Q4_K_M.gguf	1.5 GB	Mais rápido, menos capaz
TinyLlama	tinylama-1.1b-chat-Q4_K_M.gguf	0.7 GB	Muito rápido, respostas básicas
Llama 3 8B	Meta-Llama-3-8B-Instruct-Q2_K.gguf	3.5 GB	Mais capaz, mais lento

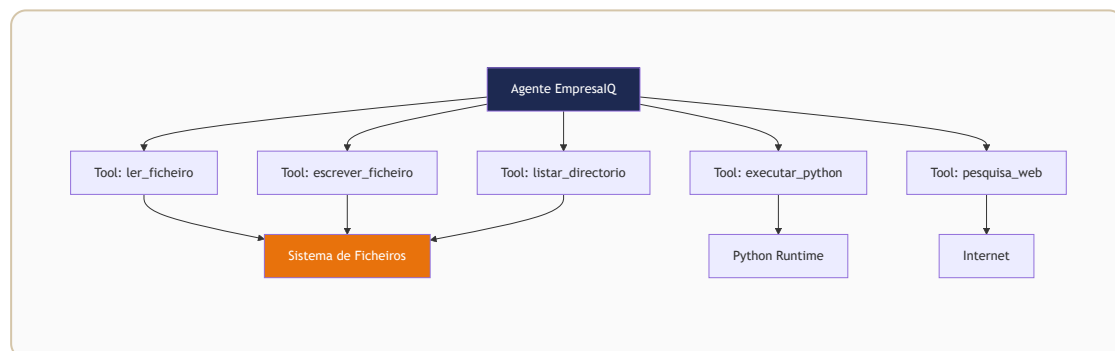
⚠ AVISO

Certifique-se de que o ficheiro `.gguf` está na mesma pasta que `agente_local.py`, ou actualize o caminho no código.

Criação das Ferramentas Inteligentes

As **ferramentas** são as capacidades que damos ao agente para interagir com o mundo real — APIs, bases de dados, ficheiros, etc.

```
mermaid graph TD
    A[Agente EmpresaIQ] --> B[Tool: consultar_ficheiro]
    A --> C[Tool: pesquisa_web]
    A --> D[Tool: calculadora]
    A --> E[Tool: base_de_dados]
    B --> F[@tool decorator LangChain]
    C --> F
    D --> F
    E --> F
    style A fill:#1D2951,color:#fff
    style F fill:#E8720C,color:#fff
```



O Ficheiro tools.py

Crie o ficheiro `tools.py` na pasta do projecto:

tools.py

```
import requests
from langchain.agents import tool

@tool
def consultar_portal_base(query_pesquisa: str) -> str:
```

```

        """Consulta contratos públicos no Portal Base do governo português."""

        url = f"https://base.gov.pt{query_pesquisa}"

        try:
            response = requests.get(url, timeout=5)

            if response.status_code == 200:
                return str(response.json()[2])

            return "Portal Base indisponível."

        except Exception as e:
            return f"Erro: {str(e)}"

@tool
def consultar_portfolio_empresaiq(servico_solicitado: str) -
> str:
    """Consulta serviços e preços do portfolio interno da EmpresaIQ."""

    base_dados_interna = {
        "software": "EmpresaIQ Core: 5.000€/ano – Gestão empresarial completa",
        "consultoria": "Consultoria IA: 120€/hora – Implementação e formação",
        "cibersegurança": "EmpresaIQ Guard: 8.500€ – Auditoria e protecção",
        "formacao": "Workshops IA: 800€/dia – Equipas até 20 pessoas",
        "desenvolvimento": "Desenvolvimento custom: orçamento sob consulta",
    }

    for chave, descricao in base_dados_interna.items():
        if chave in servico_solicitado.lower():
            return descricao

    return "Nenhum serviço encontrado. Serviços disponíveis: " + ", ".join(base_dados_interna.keys())

```

Como Funcionam as Ferramentas

O decorador `@tool` do LangChain transforma uma função Python numa ferramenta que o agente pode invocar autonomamente.

```
Agente recebe pergunta
    ↓
Analisa quais ferramentas existem
    ↓
Decide qual ferramenta usar
    ↓
Chama a ferramenta com os argumentos certos
    ↓
Recebe resultado da ferramenta
    ↓
Formula resposta final
```

Anatomia de uma Ferramenta

```
@tool                                     # ← Regista como
ferramenta LangChain
def nome_da_ferramenta(argumento: str) -> str:
    """Descrição clara do que a ferramenta faz.""" # ← O
    agente lê isto!

    # Lógica da ferramenta
    resultado = fazer_algo(argumento)

    return str(resultado)                 # ← Sempre retornar
string
```

:::important A docstring é fundamental O agente usa a **docstring** (texto entre `"""`) para decidir quando e como usar a ferramenta. Escreva descrições claras e precisas. :::

Adicionar Mais Ferramentas

Pode expandir facilmente com novas capacidades:

```
@tool
def ler_ficheiro_texto(caminho_ficheiro: str) -> str:
    """Lê o conteúdo de um ficheiro de texto local."""
    try:
        with open(caminho_ficheiro, 'r', encoding='utf-8')
as f:
            return f.read()[0:2000]  # Limita a 2000
caracteres
    except Exception as e:
        return f"Erro ao ler ficheiro: {str(e)}"

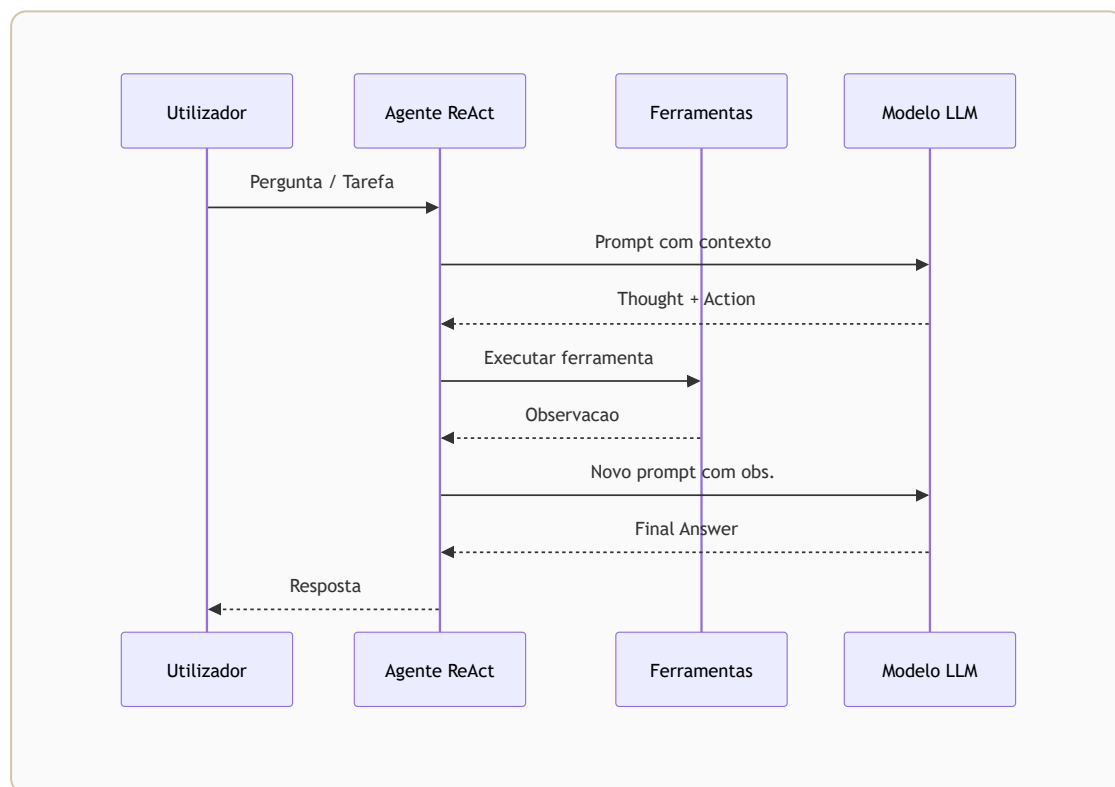
@tool
def calcular_iva(valor_sem_iva: str) -> str:
    """Calcula o valor com IVA a 23% dado um valor sem
IVA."""
    try:
        valor = float(valor_sem_iva.replace(',', '.'))
        com_iva = valor * 1.23
        return f"Valor sem IVA: {valor:.2f}€ | IVA (23%):
{valor * 0.23:.2f}€ | Total: {com_iva:.2f}€"
    except Exception as e:
        return f"Erro: {str(e)}"
```

Boas Práticas

Prática	Porquê
Sempre retornar <code>str</code>	LangChain espera strings das ferramentas
Usar <code>try/except</code>	Erros de rede/disco não devem crashar o agente
Timeout nas chamadas HTTP	Evita o agente ficar bloqueado
Docstrings claras	O agente decide qual ferramenta usar com base nelas
Nomes descritivos	<code>consultar_portal_base</code> é melhor que <code>tool1</code>

Construção do Agente Empresarial

```
mermaid sequenceDiagram actor U as Utilizador participant A as Agente ReAct participant T as Ferramentas participant L as Qwen2.5 LLM U->>A: Pergunta A->>L: Thought + Contexto L-->>A: Action + Action Input A->>T: Executar ferramenta T-->>A: Observation A->>L: Nova iteração L-->>A: Final Answer A-->>U: Resposta
```



O Ficheiro Principal — agente_local.py

agente_local.py

```
from langchain_community.llms import LlamaCpp
```

```

from langchain.agents import AgentExecutor,
create_react_agent
from langchain_core.prompts import PromptTemplate

from tools import (
    consultar_portal_base,
    consultar_portfolio_empresaiq
)

# — Carregar o Modelo


---



print("A carregar modelo Phi-3-mini... (pode demorar 10-30 segundos)")

llm = LlamaCpp(
    model_path="./Phi-3-mini-4k-instruct-Q4_K_M.gguf",
    n_ctx=2048,          # Contexto máximo (tokens)
    n_threads=4,         # Número de threads CPU
    temperature=0.1,     # Baixa = mais preciso e
    determinístico
    verbose=False        # True para ver tokens gerados em
    tempo real
)

print("Modelo carregado!")

# — Definir Ferramentas


---



tools = [
    consultar_portal_base,
    consultar_portfolio_empresaiq
]

# — Prompt do Agente


---



template = """
És o Agente EmpresaIQ – um assistente empresarial
inteligente e profissional.
Respondes sempre em português de Portugal.
Usa as ferramentas disponíveis quando necessário para
responder com precisão.

Ferramentas disponíveis:
{tools}

```

```
Nomes das ferramentas: {tool_names}
```

```
Pergunta do utilizador: {input}
```

```
Thought: {agent_scratchpad}
```

```
"""
```

```
prompt = PromptTemplate.from_template(template)
```

```
# — Criar o Agente
```

```
agent = create_react_agent(  
    llm=llm,  
    tools=tools,  
    prompt=prompt  
)
```

```
agent_executor = AgentExecutor(  
    agent=agent,  
    tools=tools,  
    verbose=True,  
    max_iterations=3,  
    handle_parsing_errors=True  
)
```

```
# — Interface Principal
```

```
if __name__ == "__main__":
```

```
    print("\n" + "="*50)  
    print("  AGENTE EMPRESAIQ – IA LOCAL")  
    print("  Modelo: Phi-3-mini Q4 | CPU Only")  
    print("="*50)  
    print("Escreva 'sair' para terminar.\n")
```

```
    pergunta = input("Pergunta: ")
```

```
    resposta = agent_executor.invoke({  
        "input": pergunta  
    })
```

```
    print("\n--- Resposta Final ---")  
    print(resposta["output"])
```

Como Executar

```
# Activar ambiente virtual primeiro
venv\Scripts\activate # Windows
source venv/bin/activate # Linux/macOS

# Correr o agente
python agente_local.py
```

O que Acontece por Dentro

Fluxo ReAct

O agente usa o padrão **ReAct** (Reasoning + Acting):

```
Pergunta: "Qual o custo da consultoria IA da EmpresaIQ?"

Thought: Preciso de consultar o portfolio da EmpresaIQ
Action: consultar_portfolio_empresaiq
Action Input: "consultoria"
Observation: "Consultoria IA: 120€/hora – Implementação e
formação"

Thought: Tenho a informação necessária
Final Answer: A consultoria IA da EmpresaIQ custa
120€/hora...
```

Parâmetros Importantes

Parâmetro	Valor	Significado
<code>n_ctx</code>	2048	Tokens máximos de contexto (pergunta + resposta)
<code>n_threads</code>	4	Threads CPU usados para inferência
<code>temperature</code>	0.1	Próximo de 0 = preciso; próximo de 1 = criativo
<code>max_iterations</code>	3	Máximo de ciclos Thought/Action antes de desistir
<code>handle_parsing_errors</code>	True	Recupera graciosamente de erros de formato

Primeiro Teste

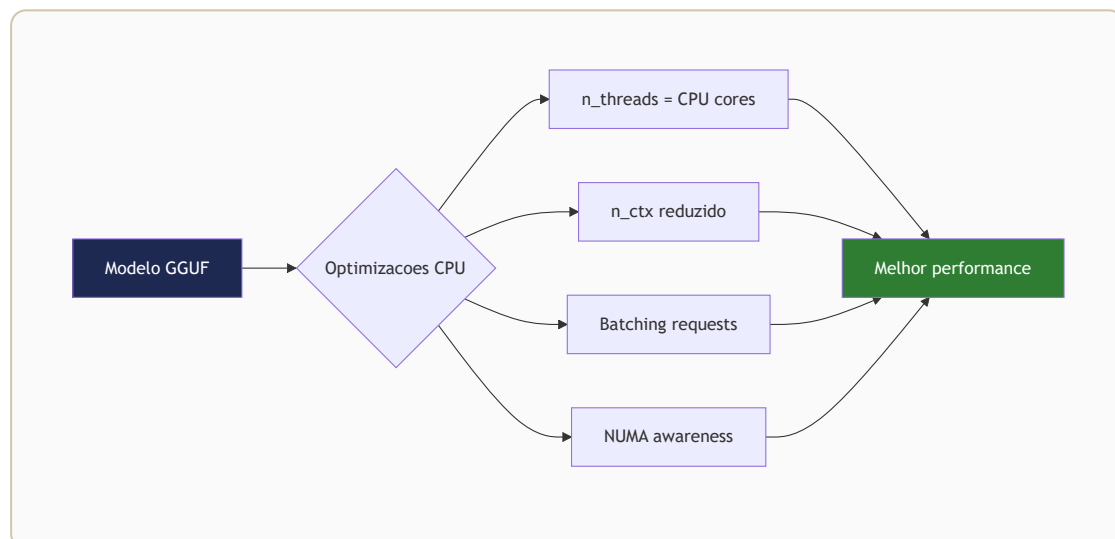
Perguntas para testar:

```
"Qual o preço do software EmpresaIQ Core?"  
"Que serviços de cibersegurança oferece a EmpresaIQ?"  
"Explica o que é quantização de modelos IA."
```

:::tip Primeira execução Na primeira execução, o modelo demora mais a responder enquanto é carregado em memória. As respostas seguintes serão mais rápidas. :::

Optimizações Críticas para CPU

```
mermaid graph TD
    A[⚡ Optimização CPU] --> B[n_threads\nNúcleos físicos]
    A --> C[n_batch 256-512\nThroughput]
    A --> D[n_ctx 2048-4096\nControlo RAM]
    A --> E[Quantização Q4\nMenos RAM]
    B --> F[Paralelismo máximo]
    C --> F
    D --> G[Velocidade de processamento]
    E --> H[Estabilidade memória]
    F --> I[Modelo mais leve]
    style A fill:#1D2951,color:#fff
```



1. Ajustar Threads ao CPU

A configuração mais impactante é o número de threads:

```
llm = LlamaCpp(  
    model_path="./modelo.gguf",  
    n_threads=4,      # ← Ajuste aqui  
    ...  
)
```


CPU	n_threads recomendado
Dual-core (2C/4T)	2
Quad-core (4C/8T)	4
Hexa-core (6C/12T)	6
Octa-core (8C/16T)	8

Como saber quantos núcleos tem

```
# Windows PowerShell
(Get-WmiObject Win32_Processor).NumberOfLogicalProcessors

# Linux
nproc

# macOS
sysctl -n hw.logicalcpu
```

DICA

Não use **todos** os threads disponíveis — deixe 1-2 para o sistema operativo funcionar sem lentidão.

2. Limitar o Contexto

O contexto (n_ctx) afecta directamente a RAM e velocidade:

```
n_ctx=2048 #  Recomendado para 8 GB RAM — rápido
n_ctx=4096 #  Aceitável — mais lento
n_ctx=8192 #  Evitar — muito lento e consome RAM
```

Regra prática: use o contexto mínimo necessário para a tarefa.

3. Temperature Baixa para Agentes

```
temperature=0.1 #  Precisão máxima – ideal para agentes
temperature=0.5 # Criatividade moderada
temperature=0.9 # Alta criatividade – respostas variadas
```

Para agentes empresariais, **temperature baixa** é sempre melhor:

- Respostas mais previsíveis e consistentes
- Seguimento de instruções mais rigoroso
- Menos "alucinações"

4. Fechar Aplicações que Consomem RAM

Antes de executar o agente:

```
✗ Fechar:
- Chrome / Edge com muitos separadores
- Discord / Teams / Slack
- Docker Desktop
- Adobe Creative Suite
- Jogos

 Manter aberto:
- Terminal / PowerShell
- VS Code (se necessário)
```

Ver consumo de RAM actual

```
# Windows PowerShell
Get-Process | Sort-Object WorkingSet -Descending | Select-Object -First 10 Name, @{N='RAM(MB)';E={
[math]::Round($_.WorkingSet/1MB,0)}}

# Linux
ps aux --sort=-%mem | head -10
```

5. Activar AVX2 (CPU moderno)

Se o seu CPU suporta AVX2 (Intel desde 2013, AMD Ryzen), recompile o llama-cpp-python com esta optimização:

```
# Windows
set CMAKE_ARGS="-DLLAMA_AVX2=on -DLLAMA_AVX=on"
pip install llama-cpp-python==0.2.76 --force-reinstall --no-cache-dir

# Linux / macOS
CMAKE_ARGS="-DLLAMA_AVX2=on -DLLAMA_AVX=on" \
pip install llama-cpp-python==0.2.76 --force-reinstall --no-cache-dir
```

Melhoria esperada: 30-50% mais rápido.

6. Configuração Completa Optimizada

llm optimizado

```
llm = LlamaCpp(
    model_path="./Phi-3-mini-4k-instruct-Q4_K_M.gguf",
    n_ctx=2048,
    n_threads=4,          # Ajuste ao seu CPU
    n_batch=512,          # Tamanho do batch de processamento
    temperature=0.1,
    repeat_penalty=1.1,   # Evita repetições nas respostas
    top_p=0.9,
    verbose=False,
    use_mmap=True,         # Memory-mapped file – carregamento
                           # mais rápido
    use_mlock=False,      # Não bloqueia RAM – deixa OS gerir
)
```

Comparação de Velocidade

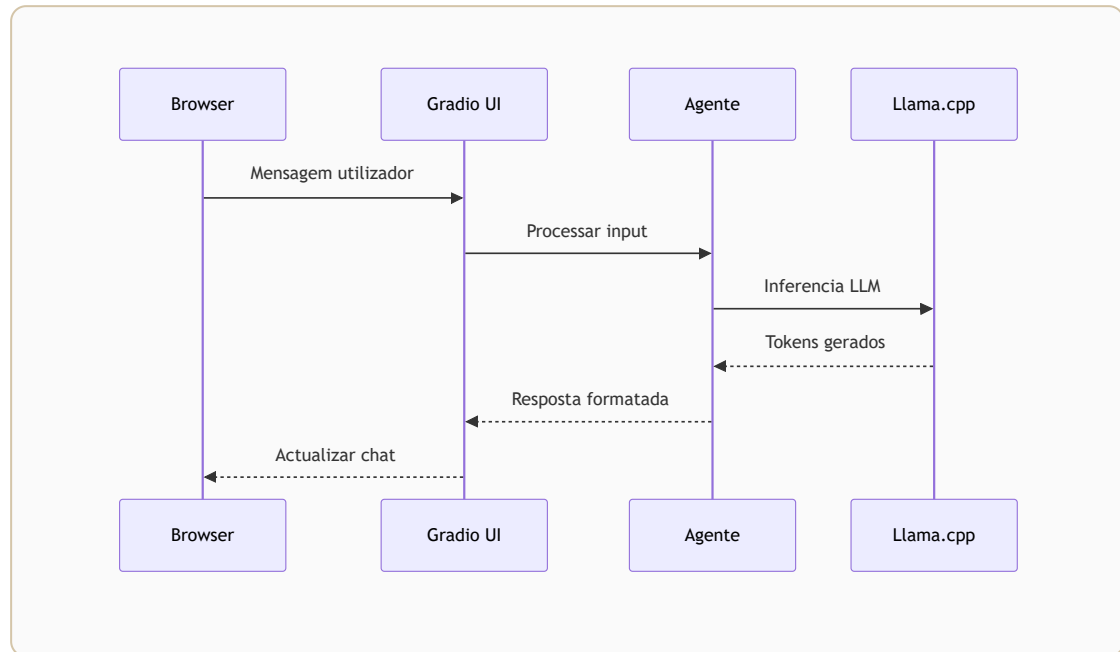
Com as optimizações aplicadas num PC típico (Intel i5, 8 GB RAM):

Configuração	Tokens/segundo
Base (sem optimizações)	~5-8 tok/s
+ n_threads correcto	~10-15 tok/s
+ AVX2 compilado	~15-25 tok/s
+ n_ctx reduzido	~20-30 tok/s

Para uma resposta de 100 tokens:

- Sem optimizações: ~15 segundos
- Com optimizações: ~4-5 segundos

Interface de Chat Local



Chat Simples em Terminal

Adicione um loop de conversação ao `agente_local.py`:

interface de chat — adicionar ao `agente_local.py`

```
if __name__ == "__main__":

    print("\n" + "="*50)
    print("  AGENTE EMPRESAIQ — IA LOCAL")
    print("  Modelo: Phi-3-mini Q4 | CPU Only")
    print("="*50)
    print("Escreva 'sair' para terminar.\n")

    while True:
        try:
            pergunta = input("Você: ").strip()

            if not pergunta:
                continue
```

```

        if pergunta.lower() in ("sair", "exit", "quit"):
            print("Até logo!")
            break

        print("Agente a processar...")

        resposta = agent_executor.invoke({
            "input": pergunta
        })

        print(f"\nAgente: {resposta['output']}\n")
        print("-" * 40)

    except KeyboardInterrupt:
        print("\nInterrompido pelo utilizador.")
        break
    except Exception as e:
        print(f"Erro: {e}")
        continue

```

Chat com Histórico de Conversação

Para o agente "lembrar" as mensagens anteriores da sessão:

chat com memória de sessão

```

from langchain.memory import ConversationBufferWindowMemory

# Memória das últimas 5 trocas
memory = ConversationBufferWindowMemory(
    k=5,
    memory_key="chat_history",
    return_messages=True
)

# Prompt actualizado com histórico
template_com_memoria = """
És o Agente EmpresaIQ.

Histórico da conversa:
{chat_history}

```

```

Ferramentas: {tools}
Nomes: {tool_names}

Pergunta actual: {input}
Thought: {agent_scratchpad}
"""

prompt_memoria =
PromptTemplate.from_template(template_com_memoria)

agent_com_memoria = create_react_agent(llm, tools,
prompt_memoria)

executor_com_memoria = AgentExecutor(
    agent=agent_com_memoria,
    tools=tools,
    memory=memory,
    verbose=False,
    max_iterations=3,
    handle_parsing_errors=True
)

```

⚠ Memória e RAM Cada mensagem guardada na memória ocupa tokens de contexto. Com `k=5` (5 trocas), o contexto pode crescer rapidamente. Em hardware limitado, use `k=3` ou `k=2`. ⚠

Interface Web Simples com Flask

Para uma interface web básica:

```
pip install flask
```

web_chat.py

```

from flask import Flask, request, jsonify,
render_template_string

app = Flask(__name__)

```

```

HTML = """
<!DOCTYPE html>
<html>
<head>
  <title>EmpresaIQ Agent</title>
  <style>
    body { font-family: Arial, sans-serif; max-width:
700px; margin: 40px auto; padding: 20px; }
    #chat { border: 1px solid #ddd; height: 400px;
overflow-y: auto; padding: 15px; margin-bottom: 10px; }
    input { width: 80%; padding: 8px; }
    button { padding: 8px 16px; background: #0078d4;
color: white; border: none; cursor: pointer; }
    .user { color: #0078d4; }
    .agent { color: #107c10; }
  </style>
</head>
<body>
  <h2>🤖 Agente EmpresaIQ</h2>
  <div id="chat"></div>
  <input id="input" type="text" placeholder="Escreva a sua
pergunta...">
  <button onclick="enviar()">Enviar</button>
  <script>
    async function enviar() {
      const input = document.getElementById('input');
      const chat = document.getElementById('chat');
      const pergunta = input.value.trim();
      if (!pergunta) return;
      chat.innerHTML += `<p class="user"><b>Você:</b>
${pergunta}</p>`;
      input.value = '';
      const r = await fetch('/chat', {method:'POST',
headers:{'Content-Type':'application/json'}, body:
JSON.stringify({pergunta})});
      const data = await r.json();
      chat.innerHTML += `<p class="agent"><b>Agente:
</b> ${data.resposta}</p>`;
      chat.scrollTop = chat.scrollHeight;
    }

    document.getElementById('input').addEventListener('keypress',
e => { if(e.key==='Enter') enviar(); });
  </script>
</body>
</html>
"""

```



```
@app.route('/')
def index():
    return render_template_string(HTML)

@app.route('/chat', methods=['POST'])
def chat():
    data = request.get_json()
    pergunta = data.get('pergunta', '')
    resposta = agent_executor.invoke({"input": pergunta})
    return jsonify({"resposta": resposta["output"]})

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=5000, debug=False)
```

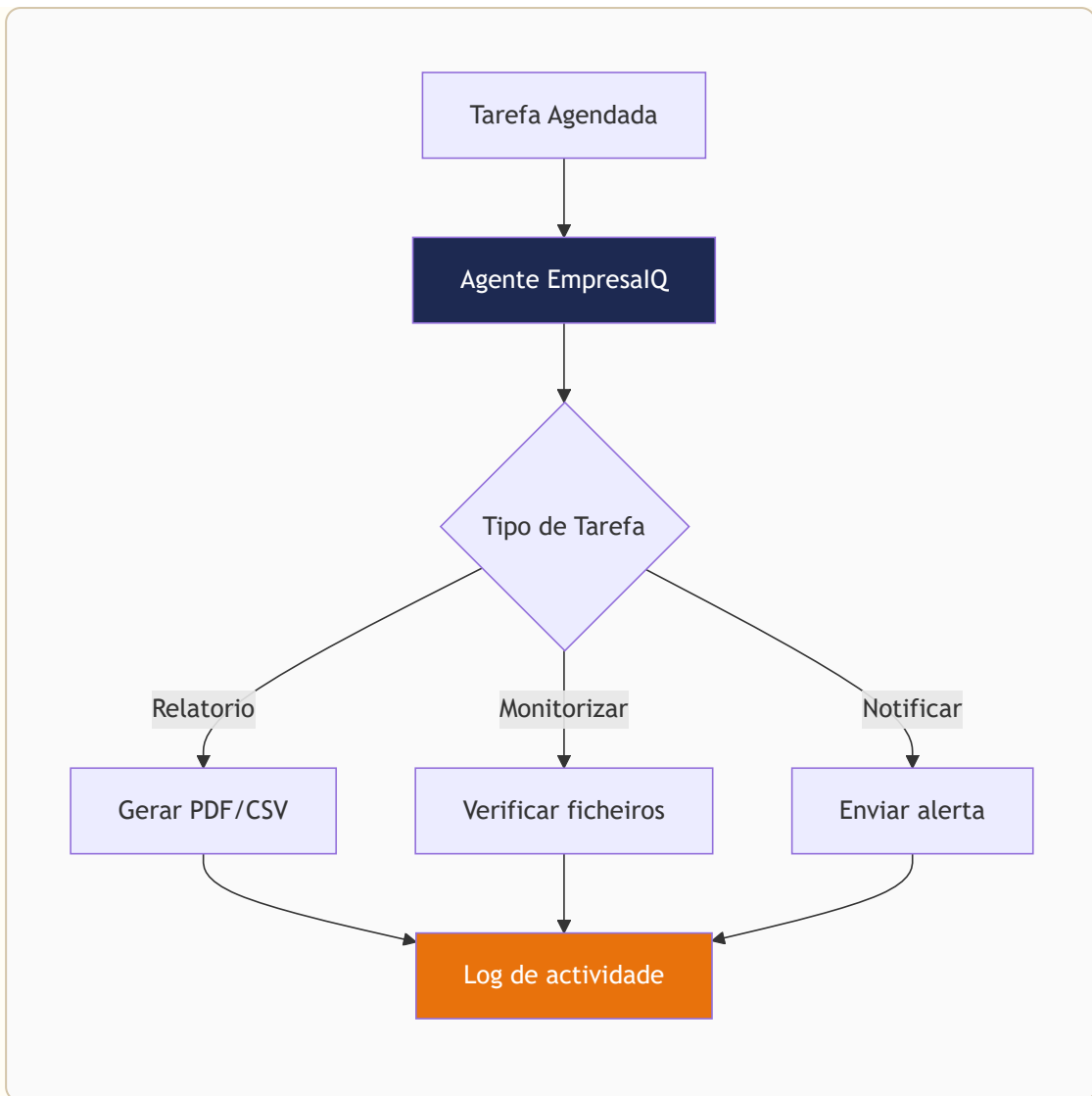
Aceda em: **http://localhost:5000**

:::info Segurança O servidor Flask está configurado apenas para `127.0.0.1` (localhost). Nunca exponha o agente diretamente à internet sem autenticação. :::

Automatização Invisível por Hora

O agente pode correr automaticamente em segundo plano, sem intervenção humana — gerando relatórios, verificando dados, enviando alertas.

```
mermaid flowchart LR
  A[🕒 Schedule / Trigger] --> B[Tarefa Automática]
  B --> C[Agente EmpresaIQ]
  C --> D{Tipo de tarefa}
  D --> E[✉️ Relatório Email]
  D --> F[📊 Análise Dados]
  D --> G[📁 Processar Ficheiros]
  E --> H[📤 Output]
  F --> H
  G --> H
  style C fill:#1D2951,color:#fff
```



Criar um Script Automatizável

Primeiro, transforme o agente num script não-interactivo:

agente_automatico.py

```
import sys
from agente_local import agent_executor
from datetime import datetime
import json

def executar_tarefa(pergunta: str, ficheiro_output: str = None):
    """Executa uma tarefa no agente e guarda o resultado."""

    print(f"[{datetime.now().strftime('%Y-%m-%d %H:%M')}] A processar: {pergunta}")
```

```

    resultado = agent_executor.invoke({"input": pergunta})
    resposta = resultado["output"]

    print(f"Resposta: {resposta}")

    if ficheiro_output:
        with open(ficheiro_output, 'a', encoding='utf-8') as
f:
            entrada = {
                "timestamp": datetime.now().isoformat(),
                "pergunta": pergunta,
                "resposta": resposta
            }
            f.write(json.dumps(entrada, ensure_ascii=False)
+ "\n")

    return resposta

if __name__ == "__main__":
    # Tarefa padrão ou argumento da linha de comandos
    tarefa = sys.argv[1] if len(sys.argv) > 1 else "Resume
os serviços disponíveis da EmpresaIQ"
    executar_tarefa(tarefa,
ficheiro_output="log_agente.jsonl")

```

Linux — Cron

```

# Abrir editor de cron
crontab -e

```

Exemplos de agendamento

```

# Correr a cada hora
0 * * * * cd /home/user/empresaiq-agent && source
venv/bin/activate && python agente_automatico.py >>
/var/log/agente.log 2>&1

# Correr todos os dias às 08:00

```

```
0 8 * * * cd /home/user/empresaiq-agent && source
venv/bin/activate && python agente_automatico.py "Gera
relatório diário de serviços" >> /var/log/agente.log 2>&1

# Correr às 2ª feira às 09:00
0 9 * * 1 cd /home/user/empresaiq-agent && source
venv/bin/activate && python agente_automatico.py "Resumo
semanal do portfolio" >> /var/log/agente.log 2>&1
```

Sintaxe Cron

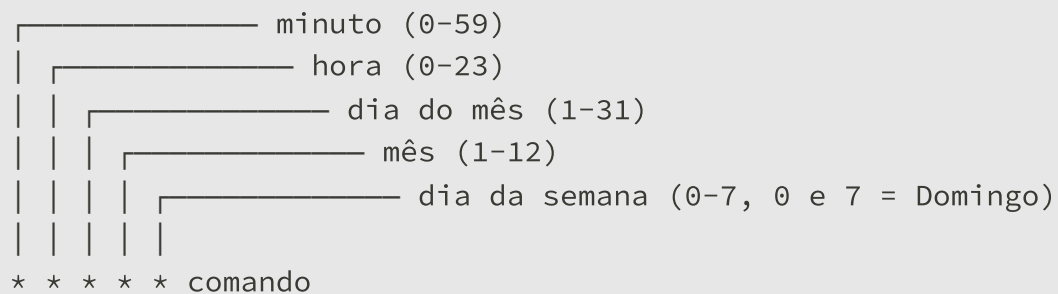


Diagrama de sintaxe Cron:

- minuto (0-59)
- hora (0-23)
- dia do mês (1-31)
- mês (1-12)
- dia da semana (0-7, 0 e 7 = Domingo)
- * * * * * comando

Windows — Agendador de Tarefas

Via PowerShell (linha de comandos)

```
# Criar tarefa agendada para correr de hora a hora
$action = New-ScheduledTaskAction `
    -Execute "python" `
    -Argument "C:\empresaiq-agent\agente_automatico.py" `
    -WorkingDirectory "C:\empresaiq-agent"

$trigger = New-ScheduledTaskTrigger -RepetitionInterval
(New-TimeSpan -Hours 1) -Once -At (Get-Date)

Register-ScheduledTask `
    -TaskName "EmpresaIQ-Agente-Automatico" `
    -Action $action `
    -Trigger $trigger `
    -Description "Agente IA EmpresaIQ – execução automática"
```

Via Interface Gráfica

1. Abra **Agendador de Tarefas** (Task Scheduler)
2. Clique em **Criar Tarefa Básica...**
3. Nome:
4. Trigger: **Diariamente** ou **Quando o computador iniciar**
5. Acção: **Iniciar um programa**
 - o Programa:
 - o Argumentos:
 - o Iniciar em:

Casos de Uso Automáticos

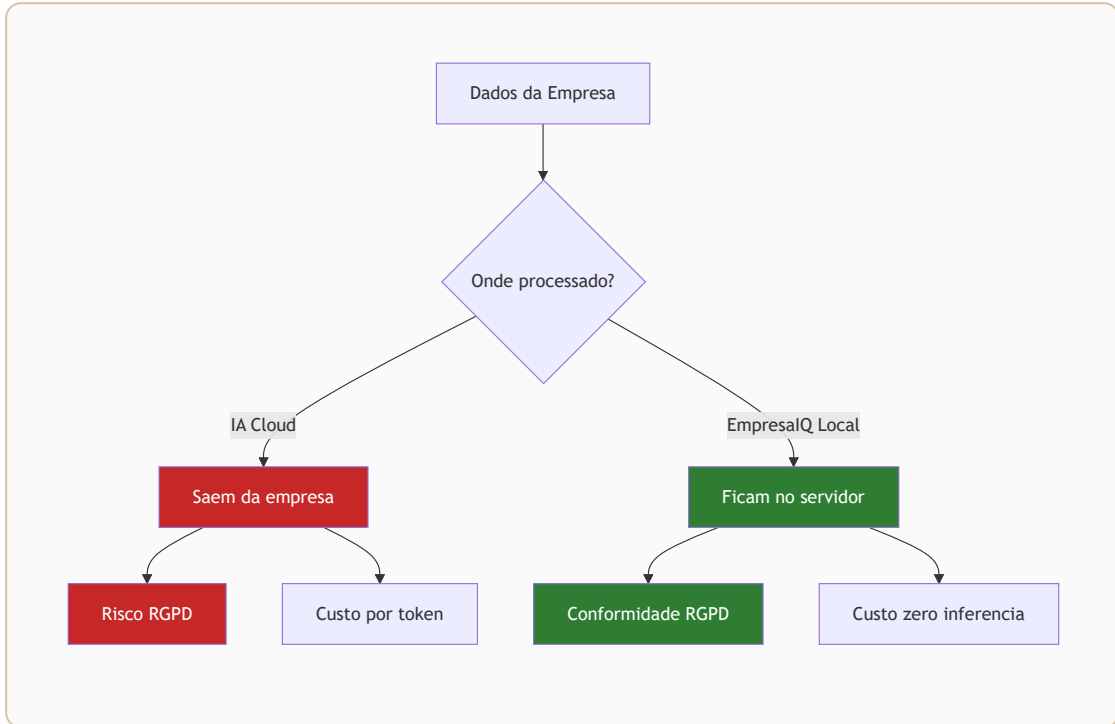
Tarefa	Agendamento	Benefício
Relatório diário de portfolio	Todos os dias 08:00	Preparação para reuniões
Verificação de contratos públicos	De 2 em 2 horas	Monitorização contínua
Resumo semanal	Segunda às 09:00	Visão geral da semana
Alerta de novos concursos	Cada 30 minutos	Oportunidades em tempo real



DICA

Guarde os resultados em ficheiros `.jsonl` (JSON Lines) — são fáceis de analisar com Python ou importar para Excel/Power BI.

Segurança e Privacidade



A Vantagem Fundamental

Com IA local, a segurança está garantida por arquitectura — não por política ou contrato.

Cloud AI:

Dados → Internet → Servidores externos → Resposta

⚠ Dados viajam fora do seu controlo

IA Local EmpresaIQ:

Dados → CPU local → Resposta

✅ Dados nunca saem do seu computador

Conformidade RGPD

O Regulamento Geral de Protecção de Dados (RGPD) exige que:

Requisito RGPD	IA Local	APIs Cloud
Dados não saem da UE	✅ Sempre	⚠️ Depende do contrato
Controlo total dos dados	✅ Sim	❌ Limitado
Direito ao esquecimento	✅ Delete local	⚠️ Complexo
Minimização de dados	✅ Controlo total	⚠️ Logs nos servidores
Avaliação de impacto (DPIA)	✅ Simples	❌ Complexa

Sectores que Beneficiam Mais

Advocacia / Notariado

Porquê IA local é essencial:

- Sigilo profissional obrigatório
- Contratos e testamentos são confidenciais
- Dados de clientes protegidos por lei

Uso prático:

- Análise de contratos internamente
- Redacção de pareceres
- Pesquisa jurídica

Contabilidade / Finanças

Porquê IA local é essencial:

- Dados fiscais e financeiros altamente sensíveis
- Sigilo bancário e fiscal
- Risco de compliance se dados saírem para fora

Uso prático:

- Análise de balanços
- Detecção de irregularidades
- Relatórios financeiros automáticos

Saúde

Porquê IA local é essencial:

- Dados de saúde são categoria especial no RGPD
- Obrigação legal de confidencialidade
- Multas enormes por violação

Uso prático:

- Triagem de sintomas
- Análise de relatórios médicos
- Apoio a diagnóstico

Sector Público

Porquê IA local é essencial:

- Dados classificados e sensíveis
- Obrigação de soberania digital
- Impossibilidade de usar cloud estrangeira para dados sensíveis

Uso prático:

- Análise de candidaturas
- Processamento de documentos internos
- Automatização de processos

Boas Práticas de Segurança

1. Isolar o Ambiente

```
# Usar sempre ambiente virtual isolado  
python -m venv venv
```

2. Não Expor o Agente à Rede

```
# CORRECTO – apenas localhost  
app.run(host='127.0.0.1', port=5000)  
  
# ERRADO – expõe a toda a rede  
app.run(host='0.0.0.0', port=5000) # ❌ Nunca sem  
autenticação
```

3. Limpar Logs Automaticamente

```
import os  
import glob  
from datetime import datetime, timedelta
```

```
def limpar_logs_antigos(pasta=".", dias=30):  
    """Remove logs com mais de X dias."""  
    limite = datetime.now() - timedelta(days=dias)  
    for ficheiro in glob.glob(f"{pasta}/*.log"):  
        if os.path.getmtime(ficheiro) < limite.timestamp():  
            os.remove(ficheiro)
```

4. Encriptar Logs Sensíveis

```
# Instalar  
pip install cryptography  
  
# Exemplo simples de encriptação de ficheiro de log
```

```
from cryptography.fernet import Fernet  
  
# Gerar chave (guardar em local seguro!)  
chave = Fernet.generate_key()  
cifra = Fernet(chave)  
  
# Encriptar  
with open('log_agente.jsonl', 'rb') as f:  
    dados = f.read()  
    dados_cifrados = cifra.encrypt(dados)  
  
with open('log_agente.enc', 'wb') as f:  
    f.write(dados_cifrados)
```

Auditoria de Acessos

Registe sempre quem perguntou o quê:

```
import logging  
  
logging.basicConfig(  
    filename='auditoria.log',  
    level=logging.INFO,  
    format='%(asctime)s - %(message)s'  
)
```

```
def registrar_acesso(utilizador: str, pergunta: str):  
    logging.info(f"Utilizador: {utilizador} | Pergunta:  
    {pergunta[:100]}")
```

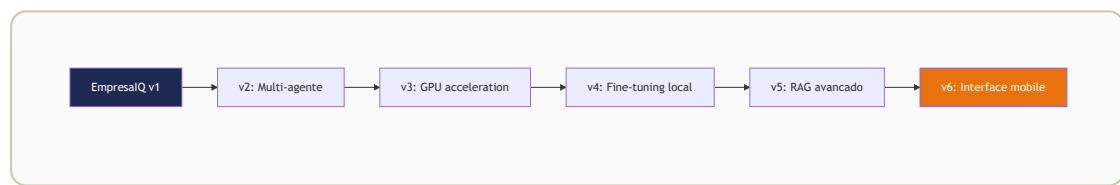
Lista de Verificação de Segurança

- ✓ Ambiente virtual isolado
- ✓ Servidor apenas em localhost
- ✓ Logs com rotação e limpeza automática
- ✓ Sem credenciais hardcoded no código
- ✓ Modelo GGUF verificado (hash SHA256)
- ✓ Actualizações regulares de dependências
- ✓ Acesso físico ao computador controlado

Melhorias Futuras

O agente base está funcional. Aqui estão os próximos passos para o tornar ainda mais poderoso.

```
mermaid graph LR
  A[Hoje\nAgente Básico] --> B[Fase 2\nRAG + Documentos]
  B --> C[Fase 3\nMulti-Agente]
  C --> D[Fase 4\nFine-Tuning Local]
  style A fill:#6B6B6B,color:#fff
  style B fill:#C0392B,color:#fff
  style C fill:#E8720C,color:#fff
  style D fill:#1D2951,color:#fff
```



1. Memória Vetorial com ChromaDB

Permita que o agente "lembre" de documentos e pesquise por similaridade semântica:

```
pip install chromadb sentence-transformers
```

memoria_vetorial.py

```
import chromadb
from chromadb.utils import embedding_functions

# Criar base de dados vetorial local
cliente = chromadb.PersistentClient(path="./db_vetorial")

# Usar modelo de embeddings local (sem internet)
embeddings =
embedding_functions.SentenceTransformerEmbeddingFunction(
```

```

    model_name="all-MiniLM-L6-v2" # ~80 MB, corre em CPU
)

colecão = cliente.get_or_create_collection(
    name="documentos_empresaiq",
    embedding_function=embeddings
)

# Adicionar documento
colecão.add(
    documents=["Contrato de prestação de serviços
EmpresaIQ..."],
    ids=["contrato_001"],
    metadatas=[{"tipo": "contrato", "data": "2026-05-29"}]
)

# Pesquisar por similaridade
resultados = coleção.query(
    query_texts=["qual é o prazo do contrato?"],
    n_results=3
)

```

Capacidades desbloqueadas:

- Responder perguntas sobre documentos internos
- Pesquisa semântica em bases de conhecimento
- "Memória" persistente entre sessões

2. Interface Web Completa com FastAPI + React

```

pip install fastapi uvicorn

```

api.py

```

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="EmpresaIQ Agent API")

```

```
class Pergunta(BaseModel):
    texto: str

@app.post("/chat")
async def chat(pergunta: Pergunta):
    resposta = agent_executor.invoke({"input":
pergunta.texto})
    return {"resposta": resposta["output"]}

# Iniciar: uvicorn api:app --host 127.0.0.1 --port 8000
```

3. OCR para Analisar Documentos PDF

```
pip install pytesseract pillow pdf2image
# Instalar Tesseract: https://github.com/UB-Mannheim/tesseract/wiki
```

```
@tool
def analisar_pdf(caminho_pdf: str) -> str:
    """Extrai e analisa o texto de um ficheiro PDF."""
    import pytesseract
    from pdf2image import convert_from_path

    imagens = convert_from_path(caminho_pdf)
    texto_completo = ""

    for imagem in imagens:
        texto_completo +=
pytesseract.image_to_string(imagem, lang='por')

    return texto_completo[:3000] # Limita ao contexto
disponível
```

4. Reconhecimento de Voz com Whisper

```
pip install openai-whisper
```

```

import whisper

modelo_whisper = whisper.load_model("small") # ~244 MB

def transcrever_audio(caminho_audio: str) -> str:
    resultado = modelo_whisper.transcribe(caminho_audio,
    language="pt")
    return resultado["text"]

# Integrar no loop de chat
import sounddevice as sd
import numpy as np
import scipy.io.wavfile as wav

def gravar_voz(segundos=5):
    """Grava áudio do microfone."""
    audio = sd.rec(int(segundos * 16000), samplerate=16000,
    channels=1)
    sd.wait()
    wav.write("temp_audio.wav", 16000, audio)
    return transcrever_audio("temp_audio.wav")

```

5. Síntese de Voz com Piper TTS

Piper é um sistema TTS local extremamente leve e com suporte a português:

```

pip install piper-tts

```

```

import subprocess

def falar(texto: str):
    """Converte texto em fala usando Piper TTS."""
    subprocess.run([
        "piper",
        "--model", "pt_PT-tugao-medium.onnx",
        "--output_file", "resposta.wav"
    ], input=texto.encode(), check=True)

# Reproduzir o áudio
subprocess.run(["aplay", "resposta.wav"]) # Linux

```

```
# subprocess.run(["start", "resposta.wav"], shell=True)
# Windows
```

6. Dashboard de Monitorização

Visualize métricas do agente com Streamlit:

```
pip install streamlit pandas
```

dashboard.py

```
import streamlit as st
import pandas as pd
import json

st.title("EmpresaIQ – Dashboard do Agente IA")

# Carregar logs
logs = []
try:
    with open("log_agente.jsonl") as f:
        for linha in f:
            logs.append(json.loads(linha))
except FileNotFoundError:
    st.warning("Sem logs registados ainda.")

if logs:
    df = pd.DataFrame(logs)
    st.metric("Total de perguntas", len(df))
    st.dataframe(df[["timestamp", "pergunta", "resposta"]])

# Iniciar: streamlit run dashboard.py
```

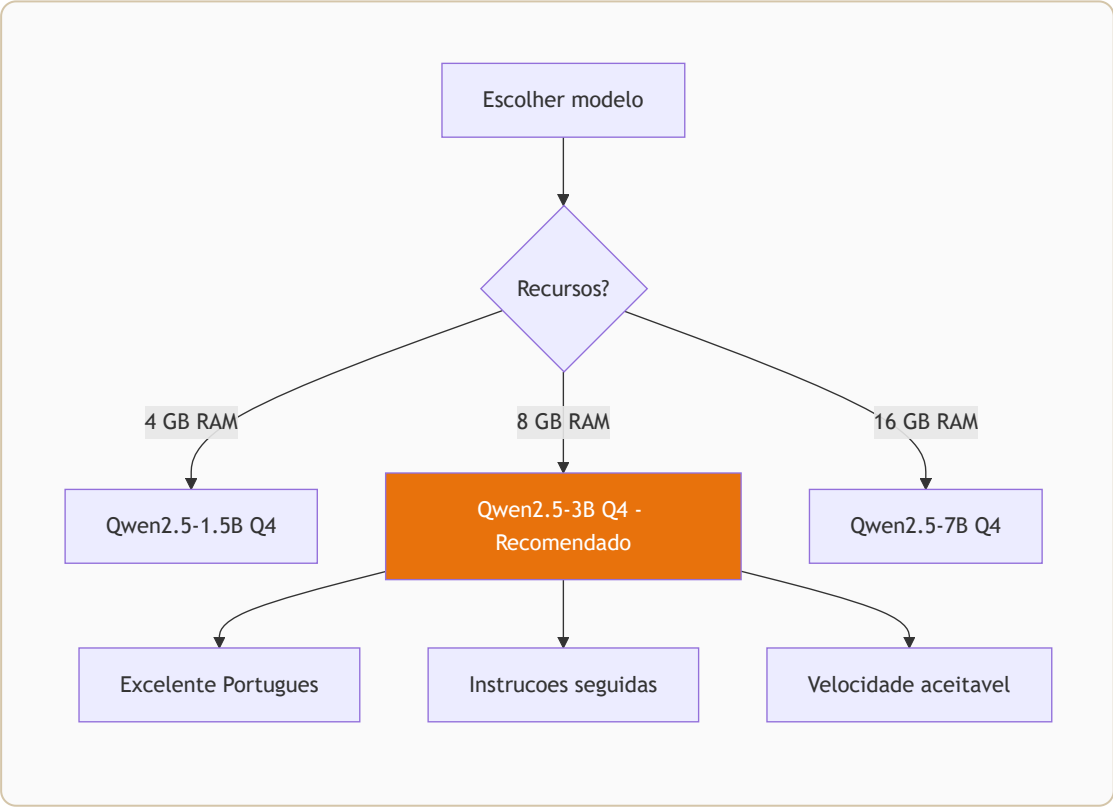
Roadmap Sugerido

```
Fase 1 (Semana 1-2): ☒ Agente base funcional
Fase 2 (Semana 3-4):  Memória vetorial com ChromaDB
```

Fase 3 (Mês 2):	Interface web FastAPI
Fase 4 (Mês 3):	OCR e análise de PDFs
Fase 5 (Mês 4):	Voz (Whisper + Piper)
Fase 6 (Mês 5-6):	Dashboard e monitorização

Capítulo 17 — Utilização do Modelo Qwen2.5 no Agente EmpresarialQ

```
mermaid graph TD
    A[Escolher Modelo Qwen2.5] --> B{RAM disponível}
    B -->|4 GB| C[Qwen2.5-1.5B Q4]
    B -->|8 GB| D[Qwen2.5-3B Q4_K_M]
    B -->|16 GB| E[Qwen2.5-7B Q4]
    C --> F[2.5-3.5 GB RAM]
    D --> F
    E --> F
    F --> G[Excelente Português]
    F --> H[Bom Tool Use]
    style D fill:#E8720C,color:#fff
```



17.1 Introdução ao Qwen

O Qwen2.5 é uma família de modelos de linguagem Open Source desenvolvida para desempenho elevado em tarefas de raciocínio, seguimento de instruções e utilização de ferramentas (tool use).

Em cenários de hardware limitado, como máquinas com 8 GB de RAM e execução apenas em CPU, o Qwen destaca-se como uma das melhores alternativas disponíveis atualmente.

Este modelo foi desenhado para ser eficiente, consistente e especialmente forte em tarefas estruturadas — tornando-o ideal para agentes inteligentes empresariais como o EmpresaIQ.

17.2 Vantagens do Qwen para Agentes Locais

Ao contrário de modelos mais antigos ou menos otimizados, o Qwen2.5 apresenta várias vantagens críticas:

- Excelente compreensão de instruções complexas
- Elevada estabilidade em fluxos ReAct (Reason + Act)
- Melhor consistência em respostas estruturadas (JSON e tool calling)
- Forte desempenho multilingue (Português, Inglês e Espanhol)
- Menor taxa de alucinação em tarefas com ferramentas externas

Estas características tornam-no particularmente adequado para sistemas de automação empresarial e agentes de decisão.

17.3 Versão Recomendada para 8 GB RAM

Para sistemas com recursos limitados, a versão ideal é:

Qwen2.5-3B-Instruct (GGUF Q4_K_M)

Consumo estimado

Recurso	Valor
RAM utilizada	~2.5 GB a 3.5 GB
CPU	Moderado
Performance	Fluida em tempo real (dependendo do processador)

Alternativas

Modelo	Velocidade	Inteligência	Indicado para
Qwen2.5-1.5B	⚡ Muito rápido	★ ★	Hardware muito limitado
Qwen2.5-3B Q4_K_M	⚡ Rápido	★ ★ ★	Recomendado
Qwen2.5-7B Q4	🐢 Moderado	★ ★ ★ ★	Próximo do limite de RAM

17.4 Integração com Llama.cpp

O Qwen pode ser facilmente executado através da biblioteca `llama-cpp-python`, mantendo compatibilidade total com arquiteturas de agentes baseadas em LangChain.

A substituição do modelo é direta:

```
model_path = "./qwen2.5-3b-instruct-q4_k_m.gguf"
```

O restante pipeline permanece inalterado, incluindo ferramentas e agente ReAct.

Exemplo completo de carregamento

```
from llama_cpp import Llama

llm = Llama(
    model_path="./qwen2.5-3b-instruct-q4_k_m.gguf",
    n_ctx=4096,
    n_threads=4,
    n_batch=256,
    verbose=False,
)
```

17.5 Ajustes Recomendados para Melhor Performance

Para garantir estabilidade e rapidez no CPU, recomenda-se:

- Definir contexto (`n_ctx`) entre 2048 e 4096
- Ajustar threads ao número de núcleos físicos do CPU
- Utilizar `temperature` entre 0.1 e 0.3
- Limitar iterações do agente (max 3 a 4 loops)

- Reduzir o tamanho das respostas das ferramentas

Configuração otimizada recomendada

```
llm = Llama(  
    model_path="./qwen2.5-3b-instruct-q4_k_m.gguf",  
    n_ctx=4096,  
    n_threads=4,          # Ajustar ao número de núcleos físicos  
    n_batch=256,  
    temperature=0.2,  
    verbose=False,  
)
```

17.6 Melhorias no Prompt para Qwen

O Qwen responde melhor a prompts diretos e estruturados.

Ao contrário de outros modelos, não necessita de prompts excessivamente longos.

Boas práticas

- Linguagem simples e directa
- Estrutura clara de Thought / Action / Observation
- Indicação explícita de resposta final em português

Exemplo de prompt eficaz

```
Você é um assistente empresarial. Responda sempre em  
português de Portugal.  
Quando usar ferramentas, siga exactamente o formato:  
Thought: [raciocínio]  
Action: [nome_ferramenta]  
Action Input: [parâmetro]  
Observation: [resultado]  
Final Answer: [resposta ao utilizador]
```

17.7 Impacto no Agente EmpresaIQ

A adoção do Qwen2.5 no agente EmpresaIQ resulta em:

- Maior precisão em decisões automatizadas
 - Melhor integração com ferramentas externas (APIs e bases de dados)
 - Redução de erros de interpretação
 - Melhor experiência conversacional
 - Maior robustez em ambientes de produção locais
-

17.8 Conclusão

O Qwen2.5 representa um salto significativo na construção de agentes inteligentes leves.

Quando combinado com Llama.cpp e uma arquitetura de ferramentas bem definida, permite criar sistemas de IA locais altamente eficientes mesmo em hardware modesto.

Para o EmpresaIQ, isto significa:

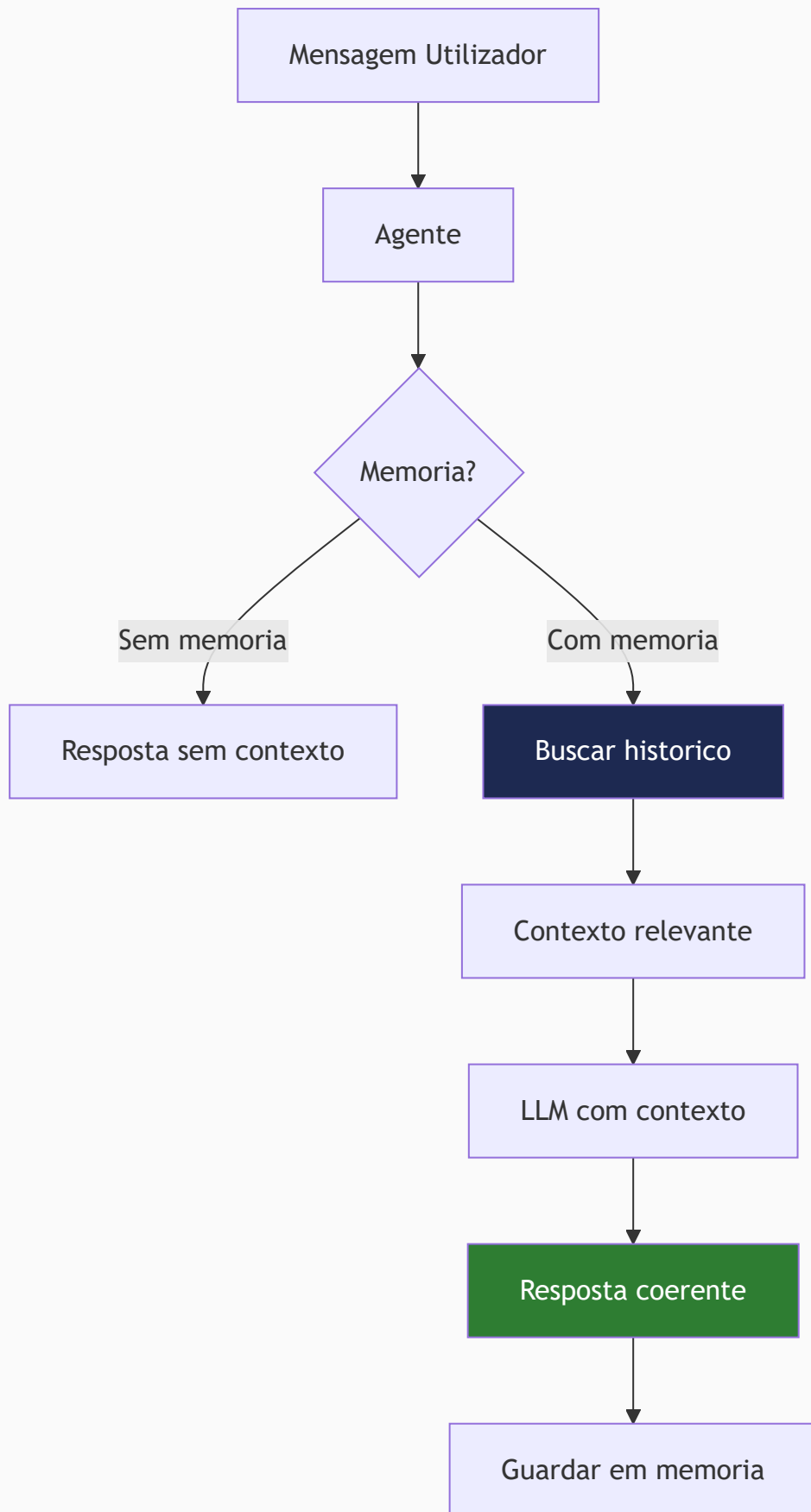
- Independência total de cloud
- Custos reduzidos
- Controle total dos dados
- Capacidade de escalar automação empresarial localmente

Este modelo torna possível aproximar agentes inteligentes de nível empresarial a qualquer computador pessoal moderno.

Capítulo 18 — Histórico e Memória Conversacional no Agente EmpresarialQ

```
mermaid graph TD
  A[Nova Pergunta] --> B[Buffer Memory\nCurto prazo k=5]
  B --> C[Summary Memory\nCompressão automática]
  C --> D[Vector Memory FAISS\nLongo prazo]
  D --> E[Contexto Combinado]
  E --> F[Qwen2.5 LLM]
  F --> G[Resposta]
  G --> B

  style E fill:#1D2951,color:#fff
  style F fill:#E8720C,color:#fff
```



18.1 Introdução

Um agente inteligente moderno não deve responder apenas com base numa pergunta isolada. Ele deve ser capaz de:

- Lembrar interações anteriores
- Manter contexto da conversa
- Evoluir decisões ao longo do tempo
- Personalizar respostas ao utilizador

Este conceito é conhecido como **Memória Conversacional (Conversation Memory)**.

No contexto do EmpresaIQ, isto é crítico para criar um assistente empresarial contínuo, e não apenas um chatbot stateless.

18.2 Problema dos Agentes Tradicionais

Por defeito, modelos como Qwen ou Phi-3:

- Não têm memória persistente
- Só "sabem" o que está no prompt atual
- Perdem contexto após cada interação

Exemplo:

Interação	Resultado
Utilizador: "Tenho 10 contratos ativos"	✅ Processado
Utilizador: "Quantos são de software?"	❌ Modelo não lembra da primeira informação

18.3 Tipos de Memória em Agentes

1. Memória de curto prazo (Context Window)

É o texto que o modelo vê no momento.

Limitação:

- Depende de `n_ctx`
- Desaparece após exceder o contexto disponível

2. Memória conversacional (Buffer Memory)

Guarda o histórico recente da conversa:

- Últimas mensagens
- Interações diretas

3. Memória persistente (Long-term Memory)

Guarda informação entre sessões:

- Base de dados
- Ficheiros

- Vector DB (FAISS / Chroma)

18.4 Memória no LangChain

O LangChain fornece sistemas prontos para memória conversacional.

ConversationBufferMemory

Guarda o histórico completo de forma simples:

```
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True
)
```

Integração no agente

```
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    memory=memory,
    verbose=True,
    max_iterations=4
)
```

Limitação: Este método cresce indefinidamente, consome RAM e não é escalável para conversas longas.

18.5 Memória Inteligente (Recomendado para EmpresaIQ)

Para sistemas reais como o EmpresaIQ, deve usar-se **memória híbrida: Buffer + Resumo + Vetores**.

ConversationSummaryMemory

O modelo resume automaticamente o histórico:

```
from langchain.memory import ConversationSummaryMemory

memory = ConversationSummaryMemory(
    llm=llm,
    memory_key="chat_history"
)
```

Vantagens:

- Reduz uso de RAM
- Mantém contexto inteligente
- Evita histórico gigante

18.6 Memória com Base Vetorial (Avançado)

O conceito mais poderoso: guardar conversas como embeddings.

Fluxo

1. Conversa ocorre
2. Texto é convertido em vetor
3. Guardado em FAISS
4. Recuperado quando necessário

Criar embeddings

```
from langchain.embeddings import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
```

Criar base de memória vectorial

```
from langchain.vectorstores import FAISS

vectorstore = FAISS.from_texts(
    ["Início da conversa EmpresaIQ"],
    embedding=embeddings
)
```

18.7 Memória Empresarial EmpresaIQ

No contexto do agente EmpresaIQ, a memória deve guardar:

- Clientes mencionados
- Contratos analisados
- Decisões anteriores
- Preferências do utilizador
- Serviços consultados

Exemplo real

Utilizador (primeira interação):

"Analisa o contrato A"

Utilizador (mais tarde):

"Compara com o anterior"

O agente deve lembrar automaticamente do "Contrato A" sem que o utilizador precise de o repetir.

18.8 Arquitectura Recomendada

```
Utilizador
  ↓
Pergunta atual
  ↓
Memória Buffer (curto prazo)
  ↓
Memória Vetorial (longa duração)
  ↓
Contexto combinado
  ↓
Qwen2.5 LLM
  ↓
Resposta final
  ↓
Atualiza memória
```

18.9 Implementação Híbrida (ideal para 8 GB RAM)

A `ConversationBufferWindowMemory` mantém apenas as últimas `k` interações, controlando o consumo de RAM:

```
from langchain.memory import ConversationBufferWindowMemory

memory = ConversationBufferWindowMemory(
```

```
k=5, # Últimas 5 interações
memory_key="chat_history",
return_messages=True
)
```

18.10 Integração Final no EmpresarialQ Agent

```
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    memory=memory,
    verbose=True,
    max_iterations=4,
    handle_parsing_errors=True
)
```

18.11 Boas Práticas

Prática	Porquê
Limitar histórico (<code>k=5</code>)	Evita sobrecarga de RAM
Usar resumos	Para conversas longas
Guardar apenas dados úteis	Não armazenar lixo textual
Separar memória por utilizador	Crucial para sistemas multi-utilizador

18.12 Arquitetura Final para EmpresaIQ

Componente	Função
Buffer Memory	Curto prazo — últimas interações
Summary Memory	Compressão automática do histórico
FAISS Memory	Longo prazo — memória persistente
Qwen2.5	Raciocínio e geração de resposta

18.13 Conclusão

Adicionar memória ao agente transforma completamente o sistema:

Sem memória	Com memória
Chatbot simples	Assistente inteligente contínuo
Respostas isoladas	Análise evolutiva
Sem contexto	Comportamento empresarial real

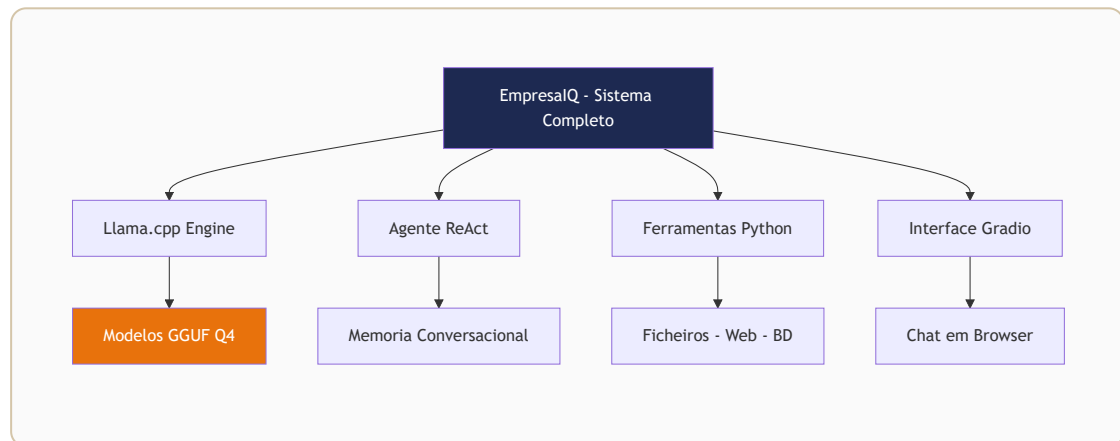
No EmpresaIQ, a memória conversacional permite:

- Análise de contratos ao longo do tempo
- Acompanhamento de clientes
- Decisões consistentes entre sessões
- Automatização real de processos

A memória é o que separa um chatbot de um verdadeiro agente inteligente.

Conclusão

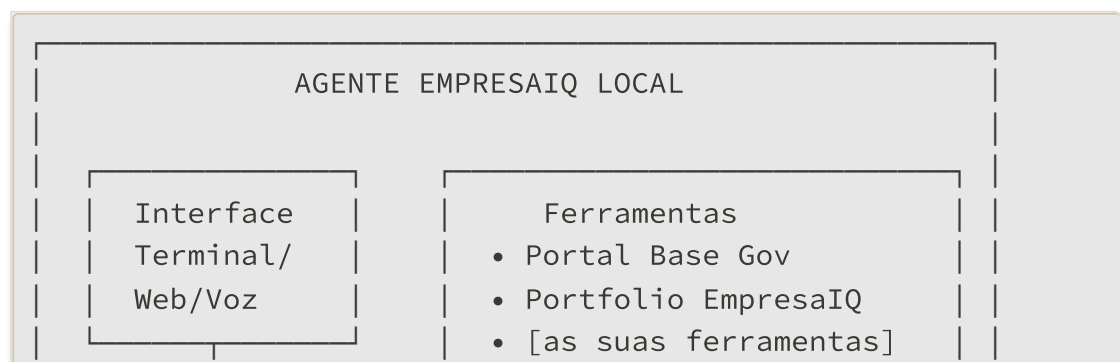
```
mermaid graph TD
  A[Computador 8 GB RAM] --> B[Llama.cpp + GGUF]
  B --> C[Qwen2.5-3B]
  C --> D[LangChain Agent]
  D --> E[Ferramentas Personalizadas]
  D --> F[Memória Conversacional]
  D --> G[Interface Gradio]
  E --> H[🧑‍💻 Agente EmpresaIQ]
  F --> H
  G --> H
  style H fill:#E8720C,color:#fff
  style C fill:#1D2951,color:#fff
```

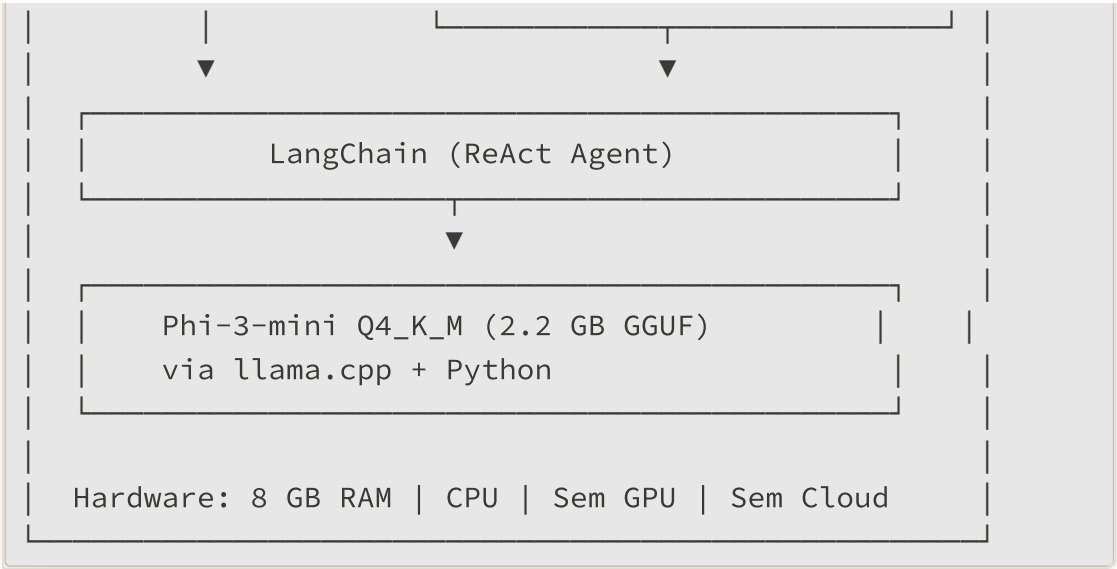


O que Construámos

Ao longo deste guia, construiu um agente inteligente completo que corre **100% localmente** num PC normal com apenas 8 GB de RAM.

Arquitectura Final





O que Aprendeu

Capítulo	Conhecimento
1-2	Porque a IA local é estratégica
3-4	Como escolher modelos para hardware limitado
5	GGUF e quantização — a base técnica
6-9	Setup completo do ambiente
10-11	Construção de ferramentas e agentes
12	Optimizações para CPU máxima
13-14	Interfaces e automatização
15	Segurança e conformidade RGPD
16	Caminhos de evolução futura

O Segredo do Sucesso

A IA local eficiente assenta em 4 pilares:

- | | |
|----------------------|-----------------------------------|
| 1. MODELO CERTO | → Phi-3-mini (3.8B parâmetros) |
| 2. FORMATO CERTO | → GGUF Q4_K_M |
| 3. MOTOR CERTO | → llama.cpp |
| 4. OPTIMIZAÇÃO CERTA | → n_threads + n_ctx + temperature |

O que Pode Fazer a Seguir

- Adicionar ferramentas específicas ao seu negócio
 - Integrar documentos com ChromaDB
 - Criar uma interface web profissional
 - Automatizar tarefas repetitivas da sua empresa
 - Expandir para voz com Whisper + Piper
-

Mensagem Final

O futuro da IA não pertence apenas à cloud.

Pertence a quem tem controlo, privacidade e autonomia.

Com este guia, a sua empresa tem agora as ferramentas para competir com soluções de IA de grandes corporações — sem depender de ninguém, sem pagar APIs e sem expor os seus dados.

O futuro também corre localmente.

Recursos Úteis

- [llama.cpp no GitHub](#)
 - [LangChain Documentação](#)
 - [Phi-3-mini no Hugging Face](#)
 - [Modelos GGUF — bartowski](#)
 - [ChromaDB](#)
 - [Piper TTS](#)
 - [Whisper OpenAI](#)
-

Guia produzido pela **EmpresaIQ** — *Inteligência Empresarial & IA*

Versão 1.0 — Maio 2026